

Indian Institute of Technology Bombay

## Assignment 1

# Building a Secure Chat Application

Course: CS6008 – Network Security

Course Instructor: Prof. Devashish Gosain

## Team Members

| Name         | Roll Number |
|--------------|-------------|
| Naveen A     | 25M1111     |
| Arun P Madhu | 25M2017     |



# Table of Contents

## Contents

|          |                                                                        |          |
|----------|------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                    | <b>1</b> |
| <b>2</b> | <b>Test Environment and VM Setup</b>                                   | <b>1</b> |
| <b>3</b> | <b>Results and Findings</b>                                            | <b>2</b> |
| 3.1      | Phase 1: Baseline Chat Application - No Security . . . . .             | 2        |
| 3.1.1    | Final Phase 1 Application . . . . .                                    | 2        |
| 3.1.2    | Wireshark Traffic Inspection . . . . .                                 | 3        |
| 3.1.3    | Protocol Description . . . . .                                         | 4        |
| 3.2      | Phase 2: Client-Server Confidentiality via Diffie-Hellman . . . . .    | 4        |
| 3.2.1    | Diffie-Hellman Key Exchange and Fingerprints . . . . .                 | 4        |
| 3.2.2    | Wireshark Traffic Inspection (Ciphertext) . . . . .                    | 6        |
| 3.2.3    | Authenticated Encryption and Tamper Detection . . . . .                | 6        |
| 3.2.4    | Man-in-the-Middle (MITM) Vulnerability Demonstration . . . . .         | 7        |
| 3.2.5    | Protocol Description and Cryptographic Choices . . . . .               | 8        |
| 3.3      | Phase 3: Server Authentication and Public Key Infrastructure . . . . . | 8        |
| 3.3.1    | Legitimate Flow and Certificate Validation . . . . .                   | 8        |
| 3.3.2    | Defeating the Phase 2 MITM Attack (Untrusted Certificate) . . . . .    | 9        |
| 3.3.3    | Proof-of-Possession and Stolen Certificates . . . . .                  | 11       |
| 3.3.4    | Cryptographic Setup . . . . .                                          | 11       |
| 3.4      | Phase 4: End-to-End Encryption (E2E) . . . . .                         | 12       |
| 3.4.1    | E2E Handshake and Key Derivation . . . . .                             | 12       |
| 3.4.2    | Server Blindness and Functional Correctness . . . . .                  | 13       |
| 3.5      | Phase 5: Perfect Forward Secrecy (PFS) via Key Rotation . . . . .      | 13       |
| 3.5.1    | Automatic Key Rotation and Collision Avoidance . . . . .               | 14       |

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| 3.5.2    | Rotation Evidence and Seamless Message Delivery . . . . .        | 14        |
| 3.5.3    | Cryptographic Analysis: The Value of Perfect Forward Secrecy . . | 15        |
| <b>4</b> | <b>Conclusion</b>                                                | <b>15</b> |
| <b>5</b> | <b>References</b>                                                | <b>16</b> |
| <b>A</b> | <b>Source Code</b>                                               | <b>17</b> |
| A.1      | Phase 1: Baseline Chat Application . . . . .                     | 17        |
| A.1.1    | Server Implementation . . . . .                                  | 17        |
| A.1.2    | Client Implementation . . . . .                                  | 20        |
| A.2      | Phase 2: Client-Server Confidentiality . . . . .                 | 23        |
| A.3      | Phase 3: Public Key Infrastructure (PKI) . . . . .               | 28        |
| A.4      | Phase 4: End-to-End Encryption . . . . .                         | 32        |
| A.5      | Phase 5: Perfect Forward Secrecy . . . . .                       | 35        |
| <b>B</b> | <b>LLM Prompts Used</b>                                          | <b>39</b> |
| B.1      | Phase 1: Basic Client-Server Communication . . . . .             | 39        |
| B.2      | Phase 2: Diffie–Hellman Key Exchange and AES-256-GCM . . . . .   | 39        |
| B.3      | Phase 3: Server Authentication via PKI . . . . .                 | 40        |
| B.4      | Phase 4: End-to-End Encryption Between Clients . . . . .         | 41        |
| B.5      | Phase 5: Perfect Forward Secrecy via Key Rotation . . . . .      | 42        |

# 1 Introduction

Secure communication is an essential requirement in modern computer networks, especially when sensitive information is exchanged between users. A basic TCP-based chat application can successfully deliver messages, but without proper security mechanisms, the communication may be vulnerable to eavesdropping, message tampering, and man-in-the-middle attacks.

This assignment focuses on building a one-to-one chat application and progressively improving its security. The application is developed in multiple phases, starting with a basic plaintext communication system and gradually introducing cryptographic techniques to protect the communication. Each phase highlights a security weakness and demonstrates how an appropriate security mechanism can address it.

The security features implemented in this assignment include Diffie–Hellman key exchange, AES-GCM encryption, server authentication using Public Key Infrastructure (PKI), and end-to-end encryption between clients. The assignment also includes the implementation of a man-in-the-middle attack to demonstrate why encryption alone is not sufficient without proper authentication.

Finally, forward secrecy is introduced through periodic key rotation, ensuring that communication does not depend on a single long-lived encryption key. The application is implemented and tested across separate virtual machines, with Wireshark captures and attack demonstrations used to verify the security properties introduced at each stage.

## 2 Test Environment and VM Setup

Our setup uses VirtualBox with two virtual machines (VMs) running Ubuntu Desktop, One VM running on Ubuntu Server and One VM running on Kali Linux. The VMs are configured with the following specifications:

Each VM is allocated 2GB of RAM and 2 CPUs. The network adapter for each VM is set to “NAT Network” mode with CIDR 10.0.2.0/24, allowing the VMs to communicate with each other while being isolated from the host machine’s network.

| VM      | IP Address | Role        | Default Gateway |
|---------|------------|-------------|-----------------|
| wadiya  | 10.0.2.10  | Server (S)  | 10.0.2.10       |
| aladeen | 10.0.2.20  | Client (C1) | 10.0.2.10       |
| nadal   | 10.0.2.30  | Client (C2) | 10.0.2.10       |
| tamir   | 10.0.2.40  | Mallory/Eve | 10.0.2.10       |

Table 1: Virtual Machine Topology



## 3 Results and Findings

### 3.1 Phase 1: Baseline Chat Application - No Security

Phase 1 was successfully implemented and tested in three development stages: basic connection testing, continuous communication using I/O multiplexing, and the final chat application. Wireshark was also used to inspect the network traffic and verify that communication occurred in plaintext.

#### 3.1.1 Final Phase 1 Application

The final application was tested with the required chat functionalities, including user registration, user selection, direct messaging, listing online users, and clean disconnection.

```
aladeen@Aladeen:~/Phase1/finalCode$ ./client
>> Welcome! Please enter your username (no spaces):
aladeen
>> Hello aladeen! Use /who to see online users.
@nadal Heyy Nadal
>> [nadal]: Haaa Supreme Leader.....
/chat nadal
[Local] Active chat partner set to: nadal
>> [nadal]: We are months away from becoming a nuclear powered nation and we are
set to testthe missile next week
It is too round on the top it needs to be pointy. Round is not scary. Pointy is
Scary
/quit
[Local] Disconnecting...
```

Figure 1: Final Phase 1 application execution on the aladeen client VM.

```
nadal@nadal:~/Phase1/finalCode$ ./client
>> Welcome! Please enter your username (no spaces):
nadal
>> Hello nadal! Use /who to see online users.
>> [aladeen]: Heyy Nadal
@aladeen Haaa Supreme Leader.....
/chat aladeen
[Local] Active chat partner set to: aladeen
We are months away from becoming a nuclear powered nation and we are set to test
the missile next week
>> [aladeen]: It is too round on the top it needs to be pointy. Round is not sca
ry. Pointy is Scary
>> [Server] Server is shutting down. Goodbye!

Server disconnected.
nadal@nadal:~/Phase1/finalCode$ █
```

Figure 2: Final Phase 1 application execution on the nadal client VM.

```
wadiya@wadiya:~/Phase1/finalCode$ ./server
Wadiya Server listening on port 8080...
Type '/quit' at any time to gracefully shut down the server.
[AUDIT LOG] Relaying plaintext from Unregistered: aladeen
[AUDIT LOG] Relaying plaintext from Unregistered: nadal
[AUDIT LOG] Relaying plaintext from aladeen: @nadal Heyy Nadal
[AUDIT LOG] Relaying plaintext from nadal: @aladeen Haaa Supreme Leader.....
[AUDIT LOG] Relaying plaintext from nadal: @aladeen We are months away from becoming a nuclear powered nation and we are
[AUDIT LOG] Relaying plaintext from aladeen: @nadal It is too round on the top it needs to be pointy. Round is not scary
aladeen disconnected.
/quit
[Server] Initiating graceful shutdown...
Disconnecting all active clients...
Wadiya Server successfully terminated.
wadiya@wadiya:~/Phase1/finalCode$ _
```

Figure 3: Server output showing client activity and plaintext message logging.

### 3.1.2 Wireshark Traffic Inspection

The network traffic was captured using Wireshark from Aladeen VM to inspect the communication between the clients and the server. Since Phase 1 does not implement any encryption, the transmitted chat messages were visible in plaintext within the captured TCP stream.

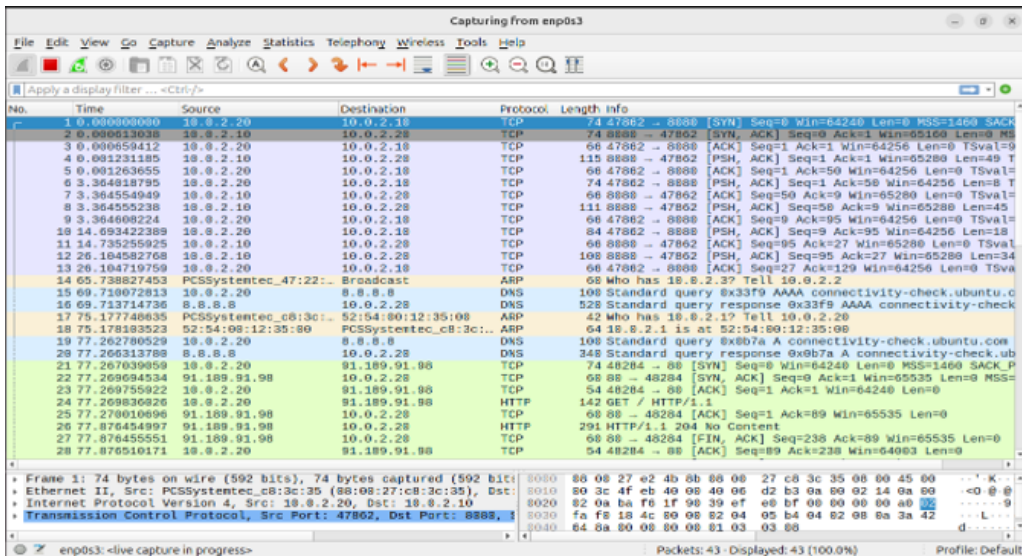


Figure 4: Wireshark capture showing the TCP traffic generated by the chat application.

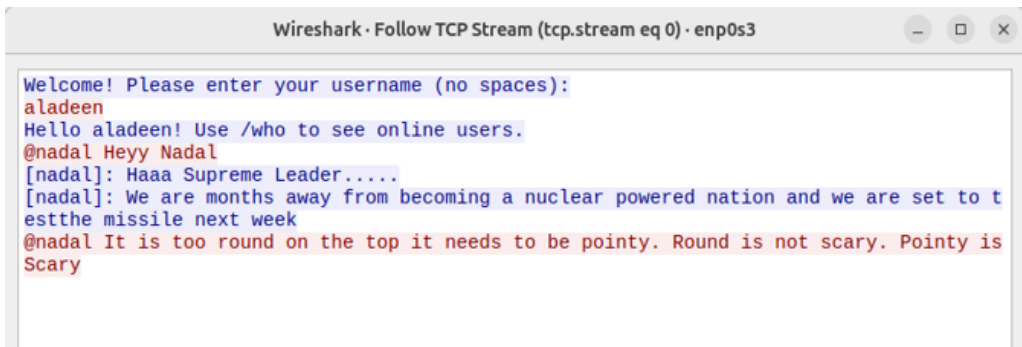


Figure 5: Inspection of the TCP stream showing chat messages in plaintext.

### 3.1.3 Protocol Description

**Message Framing and Completeness** Because TCP operates as a continuous stream of bytes rather than discrete packets, the application implements explicit delimiter-based message framing. Every outbound message is appended with a newline character (`\n`). Upon receiving incoming data, the application concatenates the raw payload into a persistent, socket-specific receive buffer.

A message is determined to be “complete” only when the application detects this `\n` delimiter within the buffer. The substring preceding the newline is then extracted and processed as a single logical unit. Any remaining bytes—representing partial fragments of a subsequent message—are preserved in the buffer until the next TCP segment arrives, ensuring data integrity regardless of network fragmentation.

**Multi-Client Routing Strategy** To support concurrent communication between more than two clients, the server operates as an application-layer router rather than a basic broadcast hub.

- **State Tracking:** The server maintains a dictionary (`std::map`) that binds active socket file descriptors to registered client usernames.
- **Targeted Parsing:** When the server receives a complete frame, it inspects the payload for the `@target_user <message>` syntax. It extracts the target’s username and the actual message content.
- **Exclusive Delivery:** The server iterates through its connection map to locate the specific socket descriptor associated with the target username. Once found, it uses the `send()` function to transmit the formatted message exclusively to that client’s socket, ensuring that private messages are not broadcast to other connected users.

## 3.2 Phase 2: Client-Server Confidentiality via Diffie–Hellman

Phase 2 successfully introduced confidentiality and integrity to the chat application. The implementation replaced plaintext communication with an encrypted channel using a Diffie-Hellman (DH) key exchange and AES-256-GCM authenticated encryption.

### 3.2.1 Diffie-Hellman Key Exchange and Fingerprints

Upon connection, each client performs an independent Diffie-Hellman key exchange with the server using modular exponentiation over a standard published prime group (RFC

3526 MODP Group 15). To verify the successful computation of the shared secret without exposing the raw key material, both the client and server print a SHA-256 hash-based fingerprint of the derived key.

As shown in the figures below, the server successfully established independent secure sessions with both clients. The server's fingerprint for Nadal's session matches Nadal's local fingerprint, and the fingerprint for Aladeen's session matches Aladeen's local fingerprint.

```
wadiya@wadiya:~/Phase2$ ./server
Secure Server listening on port 8080...
New connection. Awaiting DH Key...
[Session 4] DH Exchange Complete. Fingerprint: d3b17430f7ea0906
User registered as: nadal
New connection. Awaiting DH Key...
[Session 5] DH Exchange Complete. Fingerprint: 371aa218d76634d7
User registered as: aladeen
Client disconnected.
Client disconnected.
```

Figure 6: Server output showing independent DH key exchanges and distinct session fingerprints for nadal and aladeen.

```
Connected. Initiating Diffie-Hellman Handshake...
DH Key Exchange Complete. Fingerprint: d3b17430f7ea0906
Secure Channel Established via AES-256-GCM.
Please enter your username to register: nadal
>> Hello nadal! Registration successful.
/chat aladeen
[Local] Active chat partner set to: aladeen
Haa Supreme Leader
We are months away from becoming a nuclear powered nation, and we are about to t
est the missile next week
>> [aladeen]: It is too round on the top it needs to be pointy. Round is not sca
ry. Pointy is scary
>> [aladeen]: This will put a smile on the faces of the enemy
/quit
```

Figure 7: Nadal client output displaying the DH fingerprint matching the server's corresponding session.

```

aladeen@Aladeen:~/Phase2$ ./client
Connected. Initiating Diffie-Hellman Handshake...
DH Key Exchange Complete. Fingerprint: 371aa218d76634d7
Secure Channel Established via AES-256-GCM.
Please enter your username to register: aladeen
>> Hello aladeen! Registration successful.
/chat nadal
[Local] Active chat partner set to: nadal
>> [nadal]: Haa Supreme Leader
>> [nadal]: We are months away from becoming a nuclear powered nation, and we are about to test the missile next week
It is too round on the top it needs to be pointy. Round is not scary. Pointy is scary
This will put a smile on the faces of the enemy
/quit

```

Figure 8: Aladeen client output displaying the DH fingerprint matching the server's corresponding session.

### 3.2.2 Wireshark Traffic Inspection (Ciphertext)

To verify confidentiality, the network traffic was captured again using Wireshark. Unlike Phase 1, the TCP stream now contains unintelligible ciphertext. All application traffic, including the initial login and registration exchanges, is now securely encrypted in transit.

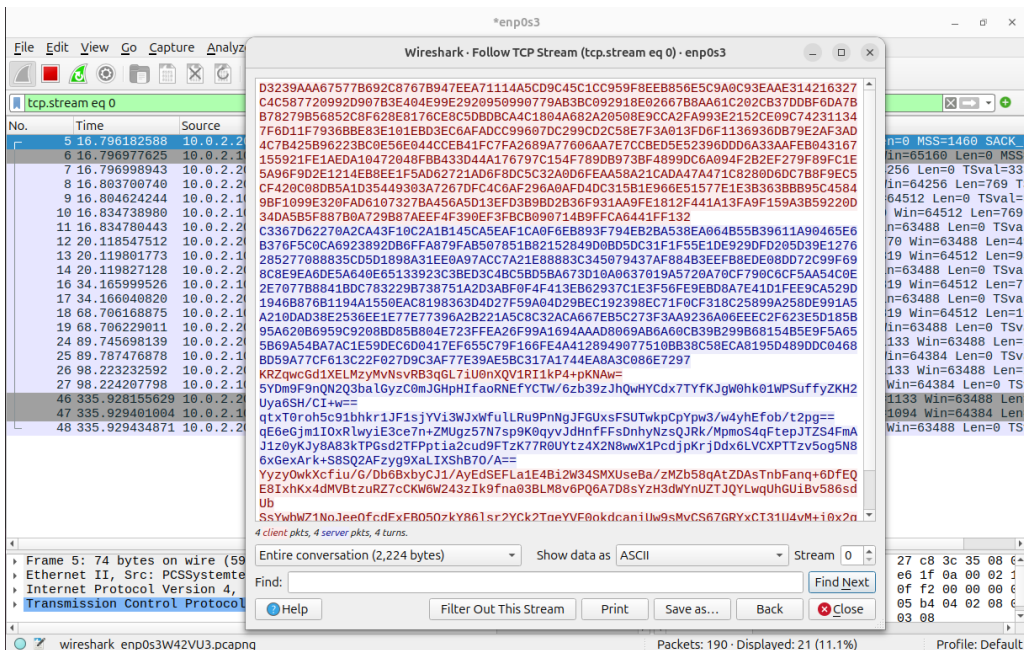


Figure 9: Inspection of the TCP stream showing that all application traffic is now encrypted base64 ciphertext.

### 3.2.3 Authenticated Encryption and Tamper Detection

The application utilizes AES-GCM (Galois/Counter Mode), which provides both confidentiality and data origin authentication. To demonstrate this capability, the payload was

intentionally modified by altering a single byte of an outbound encrypted message. Upon receiving the tampered payload, the receiving client correctly rejected the message, throwing a decryption/authentication error rather than silently processing corrupted plaintext.

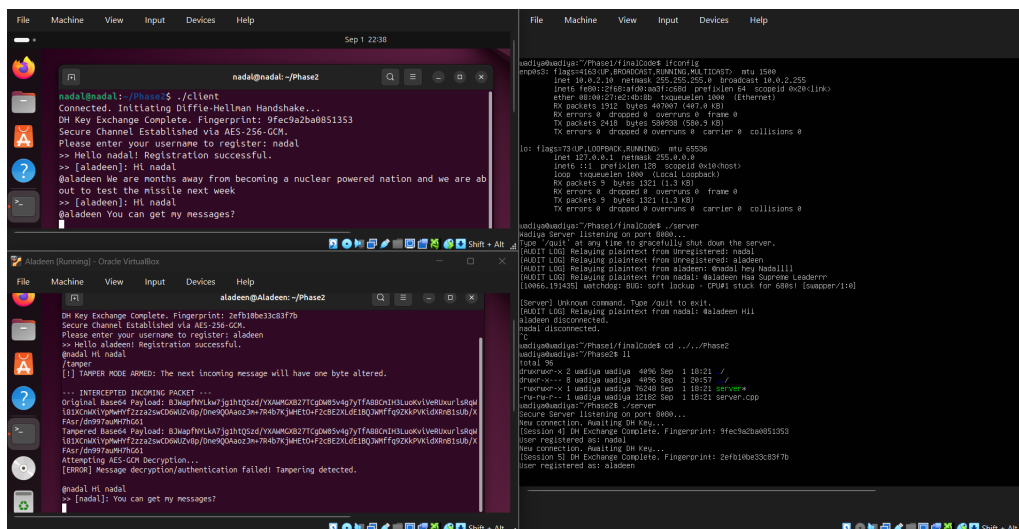


Figure 10: Demonstration of AES-GCM tamper detection rejecting a maliciously altered byte in the payload.

### 3.2.4 Man-in-the-Middle (MITM) Vulnerability Demonstration

While Phase 2 secures the connection against passive eavesdroppers, the anonymous Diffie-Hellman exchange lacks identity verification. A Man-in-the-Middle (MITM) proxy can successfully intercept the connection, perform independent DH exchanges with both the client and server, and silently decrypt, log, and re-encrypt the traffic without triggering cryptographic errors on either side.

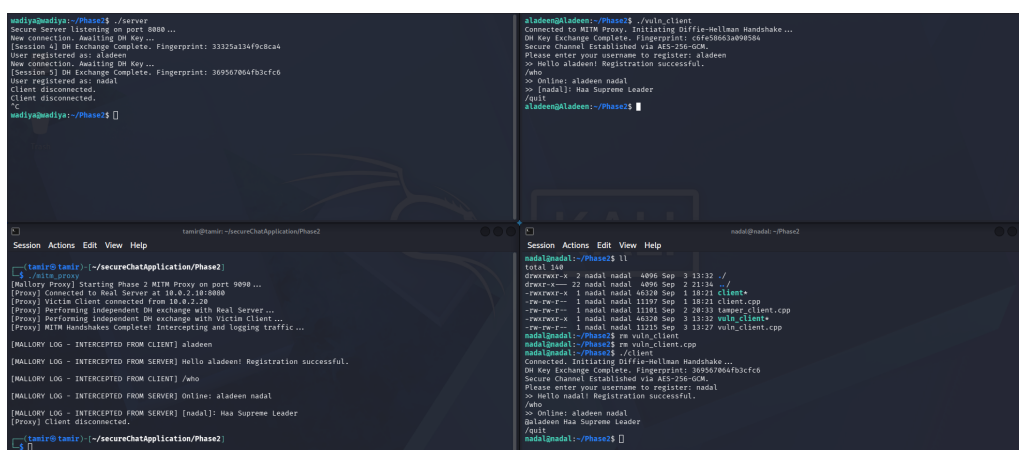


Figure 11: Execution of a MITM proxy successfully intercepting and reading secure chat application traffic.



### 3.2.5 Protocol Description and Cryptographic Choices

**Key Derivation Function (Hash Application)** After the raw Diffie-Hellman shared secret ( $g^{ab} \pmod{p}$ ) is computed, it is strictly forbidden to use it directly as the symmetric AES encryption key. Instead, the raw secret is passed through a cryptographic hash function (SHA-256) to derive the final symmetric key.

**Why hash the key?** The raw shared secret generated by Diffie-Hellman is a mathematical structure (a large integer modulo  $p$ ) that does not possess uniform randomness. Its bit distribution can contain biases, algebraic structures, or leading zeros depending on the parameters used. A symmetric encryption algorithm like AES requires a key that is a completely uniformly distributed, pseudo-random bit string. Passing the raw DH secret through a cryptographic hash function acts as a Key Derivation Function (KDF): it condenses the entropy of the large integer, eliminates mathematical biases, and produces a strong, uniformly random fixed-length key suitable for AES-256.

## 3.3 Phase 3: Server Authentication and Public Key Infrastructure

To address the Man-in-the-Middle (MITM) vulnerability identified in Phase 2, Phase 3 introduces a Public Key Infrastructure (PKI). This phase implements strict X.509 certificate validation and a cryptographic Proof-of-Possession (PoP) challenge-response mechanism to mathematically bind the server's identity to a Trusted Certificate Authority (CA) before any Diffie-Hellman parameters are exchanged.

### 3.3.1 Legitimate Flow and Certificate Validation

In the standard operational flow, the server immediately transmits its CA-signed certificate upon a new client connection. The client verifies the certificate's signature against its locally stored trusted Root CA (`ca_cert.pem`), checks the validity dates, and ensures the Common Name (CN) matches the expected server IP address. Following successful validation, the client issues a 32-byte cryptographic challenge, which the server signs using its private key to prove ownership.

As shown below, the legitimate flow executes successfully. The clients (Aladeen and Nadal) validate the certificate, confirm the proof-of-possession, and then complete the Diffie-Hellman key exchange, resulting in identical session fingerprints on both the client and server sides.

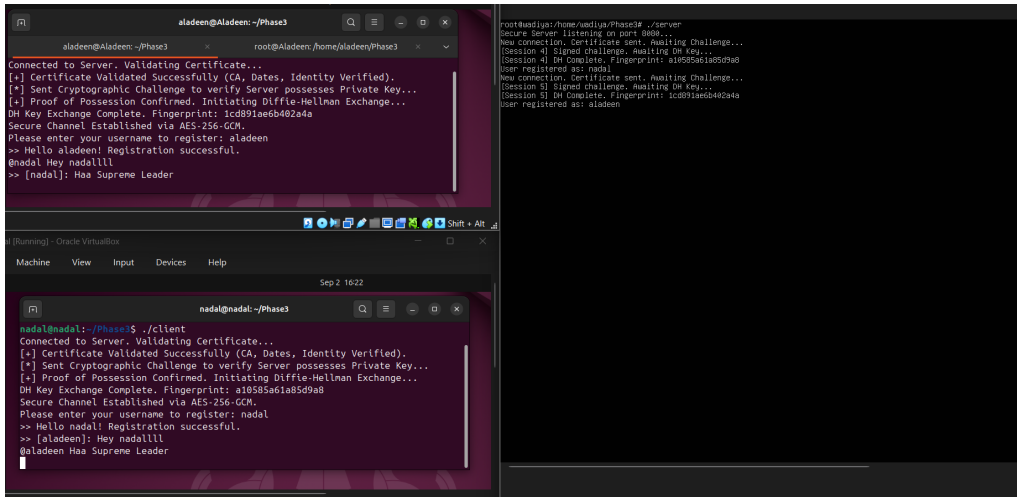


Figure 12: Successful legitimate flow showing certificate validation, PoP confirmation, and matching DH fingerprints between clients and the server.

Wireshark captures of the initialization phase confirm the transmission of the X.509 certificate in plaintext (Base64 PEM format) prior to the establishment of the encrypted tunnel.

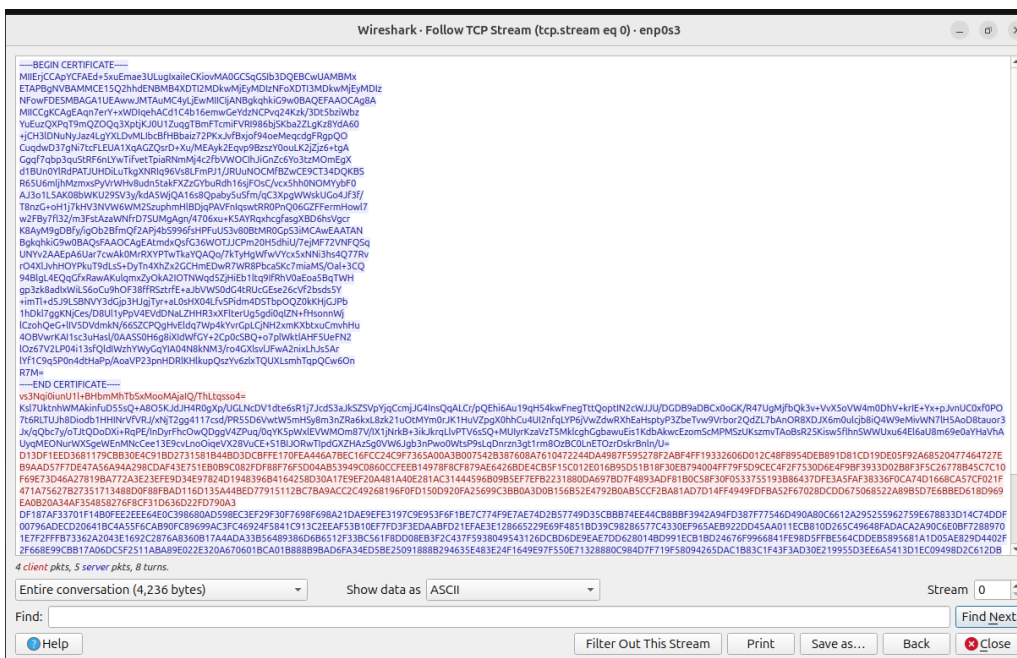


Figure 13: Wireshark TCP stream inspection showing the transmission of the server's X.509 certificate.

### 3.3.2 Defeating the Phase 2 MITM Attack (Untrusted Certificate)

To verify the PKI defense, the Phase 2 MITM proxy was re-run against the Phase 3 client. Because Mallory does not possess the Trusted CA's private key, she cannot generate a

validly signed certificate for the target IP. When Mallory attempts to present a self-signed or forged certificate, or routes traffic such that the CN does not match, the client's strict X509\_verify\_cert checks fail. The client detects the anomaly and aggressively aborts the connection before any keys are exchanged.

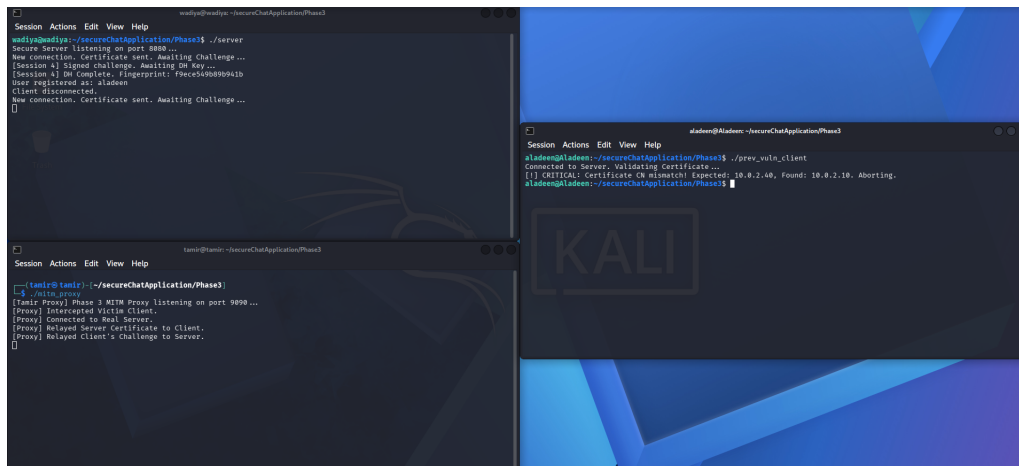


Figure 14: The MITM proxy attack fails as the client detects a certificate validation error (CN mismatch/untrusted CA) and aborts the connection.

To confirm this defense holds under real-world network hijacking conditions, the attack was repeated using a fully transparent ARP-spoofed setup. Despite Mallory seamlessly redirecting the packets via ‘iptables’ and ARP poisoning, the cryptographic mathematical chain of trust remains broken, resulting in a critical failure on the client side.

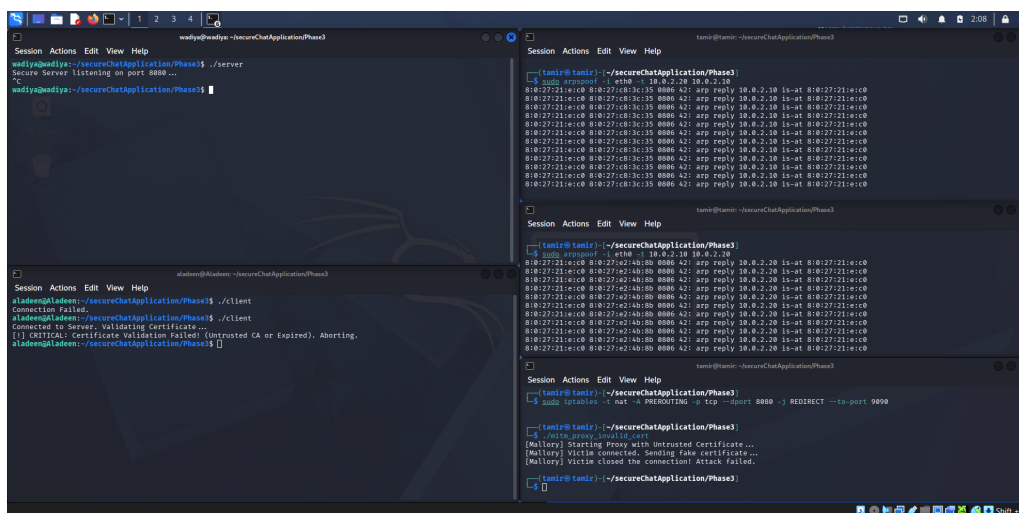
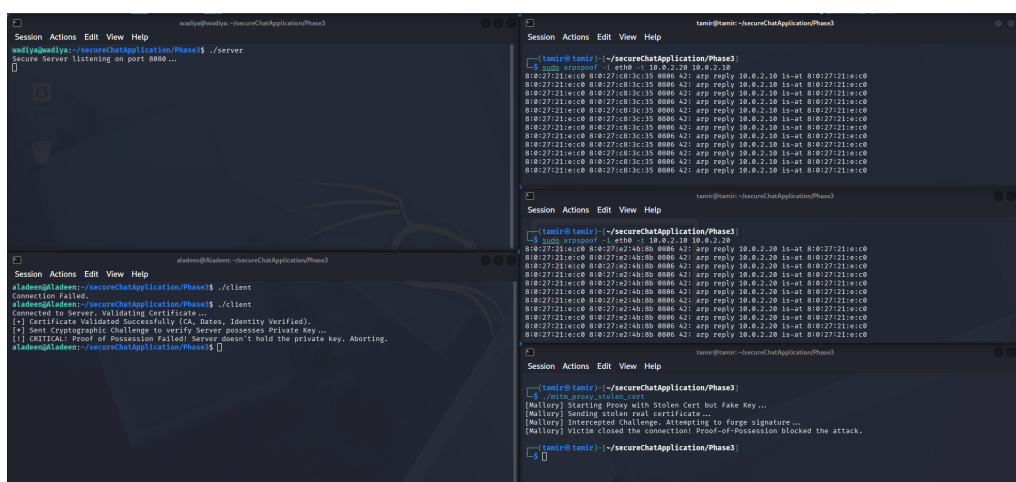


Figure 15: Transparent MITM attack via ARP spoofing defeated. The client correctly rejects Mallory's untrusted certificate under realistic network redirection.

### 3.3.3 Proof-of-Possession and Stolen Certificates

A secondary attack vector involves an attacker stealing the legitimate `server_cert.pem` (which is public) but failing to compromise the corresponding `server_key.pem`. In this scenario, Mallory passes the initial CA validation because the certificate is genuinely signed. However, when the client sends the random 32-byte cryptographic challenge, Mallory attempts to sign it with her own private key. The client verifies the signature using the public key embedded in the real certificate, detects the forgery, and terminates the connection.



```
Session Actions Edit View Help
wally@wally:~/secureChatApplication/Phase3$ ./server
Secure Server listening on port 8888...

Session Actions Edit View Help
tami@tami:~/secureChatApplication/Phase3$ ./client
[+] Certificate Validated Successfully (CA, Dates, Identity Verified).
[+] Sent Cryptographic Challenge to verify Server possesses Private Key...
[+] CERTIC: Proof of Possession failed! Server doesn't hold the private key. Aborting.
wally@wally:~/secureChatApplication/Phase3$
```

Figure 16: MITM failure using a stolen certificate. The client successfully validates the certificate but rejects the connection due to a Proof-of-Possession signature failure.

### 3.3.4 Cryptographic Setup

**OpenSSL CA and Certificate Generation** To facilitate this phase, a custom Certificate Authority was established. The following OpenSSL commands were executed to generate the required cryptographic assets, strictly binding the server's IP address ('10.0.2.10') to the certificate's Common Name (CN):

# 1. Generate the Trusted CA Private Key and Root Certificate

```
openssl genpkey -algorithm RSA -out ca_key.pem
openssl req -x509 -new -nodes -key ca_key.pem -sha256 -days 3650 \
-out ca_cert.pem -subj "/CN=TrustedCA"
```

# 2. Generate the Server Private Key and Certificate Signing Request (CSR)

```
openssl genpkey -algorithm RSA -out server_key.pem
openssl req -new -key server_key.pem -out server.csr -subj "/CN=10.0.2.10"
```

# 3. Sign the Server CSR using the Trusted CA

```
openssl x509 -req -in server.csr -CA ca_cert.pem -CAkey ca_key.pem \
    -CAcreateserial -out server_cert.pem -days 365 -sha256
```

**Remaining Vulnerability (Protocol Disconnect)** While Phase 3 successfully thwarts basic MITM attempts, the protocol design harbors a critical flaw: the Proof-of-Possession (authentication) is not cryptographically bound to the Diffie-Hellman key exchange. A sophisticated active proxy can bypass this phase by acting as a transparent relay during the certificate and challenge-response steps, and subsequently stepping in to intercept the unsigned Diffie-Hellman public parameters. Securing the DH parameters directly (Phase 4) is required to achieve complete MITM immunity.

### 3.4 Phase 4: End-to-End Encryption (E2E)

To provide absolute privacy between users, Phase 4 introduces an End-to-End (E2E) encryption protocol. This mechanism establishes a shared cryptographic secret directly between two clients ( $C_1 \leftrightarrow C_2$ ), completely independent of the server. The server acts exclusively as a blind message relay. This E2E encryption functions as an additional, inner layer of encryption wrapped inside the existing Phase 3 client-server secure tunnel.

#### 3.4.1 E2E Handshake and Key Derivation

The end-to-end session is manually triggered by the user via the `/e2e <username>` command. Upon triggering, the initiating client generates an ephemeral Diffie-Hellman key pair and transmits the public parameters to the target client.

To ensure the server's existing relay logic did not need to be modified, the application utilizes a wire-level tagging convention. Protocol messages are prepended with specific prefixes (e.g., `_E2E_INIT_`, `_E2E_ACK_`, and `_E2E_MSG_`). The server routes these messages using its standard Phase 1 `@username` routing logic, remaining entirely unaware of the underlying cryptographic exchange.

As demonstrated in the client terminals in Figure 17, both clients successfully perform the independent E2E DH handshake. The successful key derivation is verified by independently computed SHA-256 fingerprints printed on both  $C_1$  (Aladeen) and  $C_2$  (Nadal) consoles, confirming they share identical secret keys without server involvement.

### 3.4.2 Server Blindness and Functional Correctness

A critical requirement of Phase 4 is ensuring that the server cannot read the inner chat content, contrasting with Phase 2/3 where the server decrypted client messages to route them and could log the plaintext.

The server logs (displayed in the right pane of Figure 17) demonstrate that the server correctly identifies the routing metadata (e.g., `nadal1`  $\rightarrow$  `aladeen`) to deliver the message. However, the payload logged by the server consists entirely of opaque, unreadable Base64 ciphertext appended to the wire-level tags. The server cannot derive the  $C_1 \leftrightarrow C_2$  shared key, guaranteeing zero-knowledge message relaying.

Finally, functional correctness is confirmed on the client side. The encrypted payloads relayed by the server are successfully received, decrypted using the inner E2E symmetric key, and displayed to the users with an (E2E) tag, ensuring seamless and highly secure communication.

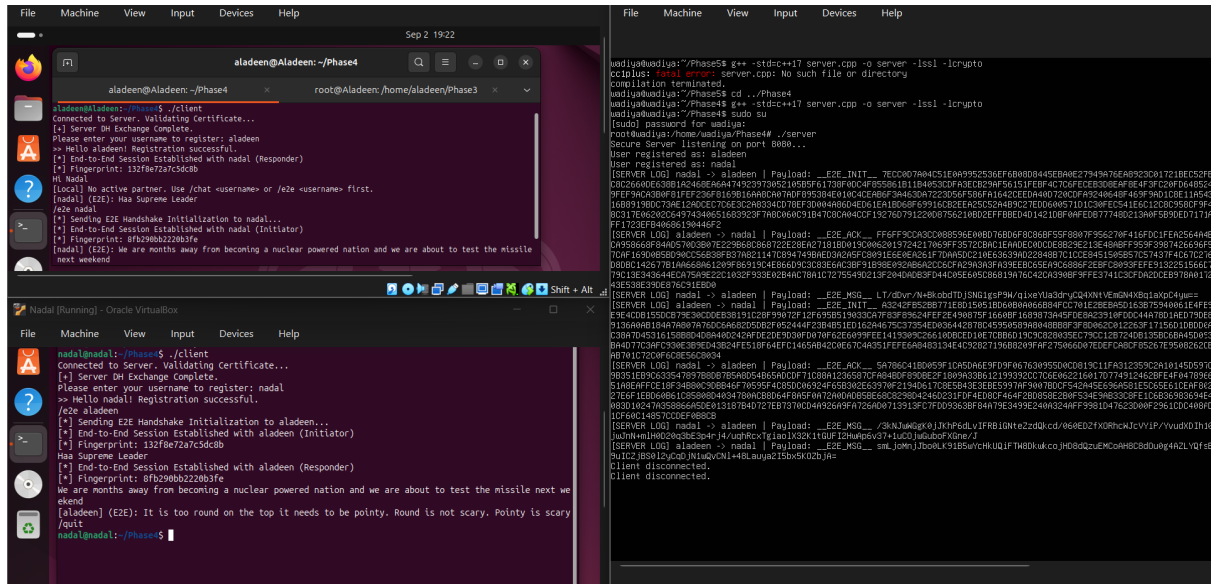


Figure 17: End-to-End Encryption execution. The left panes show successful E2E handshakes, matching key fingerprints, and correct message decryption on both clients. The right pane shows the server operating as a blind relay, logging only routing metadata and opaque E2E ciphertext.

## 3.5 Phase 5: Perfect Forward Secrecy (PFS) via Key Rotation

To enhance the security of the end-to-end encrypted session established in Phase 4, Phase 5 introduces Perfect Forward Secrecy (PFS). This is achieved by implementing an automatic, timer-based key renegotiation mechanism that limits the lifespan of any single symmetric key, explicitly discarding old key material once a new key is successfully established.



### 3.5.1 Automatic Key Rotation and Collision Avoidance

The E2E session was extended with a background timer that automatically triggers a new Diffie-Hellman key exchange every 60 seconds as long as the session remains active. Each renegotiation generates fresh ephemeral private/public parameters, guaranteeing that the new shared secret is mathematically independent of the previous one.

Because both clients operate on independent 60-second timers, there is a high probability of a race condition (a “glare” state) where both clients attempt to initiate a key rotation simultaneously. To prevent the clients from deriving conflicting keys, a deterministic tie-breaking mechanism was implemented:

- **Lexicographical Tie-Breaker:** If a client receives an inbound `E2E_REKEY_INIT` request while currently waiting for a response to its own rekey request, it compares its username with the peer’s username lexicographically.
- **Resolution:** The client with the alphabetically “higher” username ignores the incoming request and waits for its own to be acknowledged. The client with the “lower” username drops its own initiated request and immediately processes the peer’s request, acting as the responder. This ensures both clients seamlessly converge on a single, shared Diffie-Hellman exchange.

### 3.5.2 Rotation Evidence and Seamless Message Delivery

The protocol’s key rotation logic was verified by monitoring the active session across multiple rotation cycles. As shown in Figure 18, both the initiating and responding clients log the SHA-256 fingerprint of the newly derived key alongside a timestamp.

Over the course of multiple 60-second intervals, the logs demonstrate that:

1. The fingerprint changes completely upon every rotation, confirming a genuinely new key is generated.
2. Both clients independently derive and print the exact same fingerprint after the rotation completes.

Furthermore, ongoing chat operations are not disrupted by this background process. Messages sent immediately before, during, and after a rotation are correctly encrypted with the actively agreed-upon key, successfully relayed by the blind server, and correctly decrypted by the recipient.

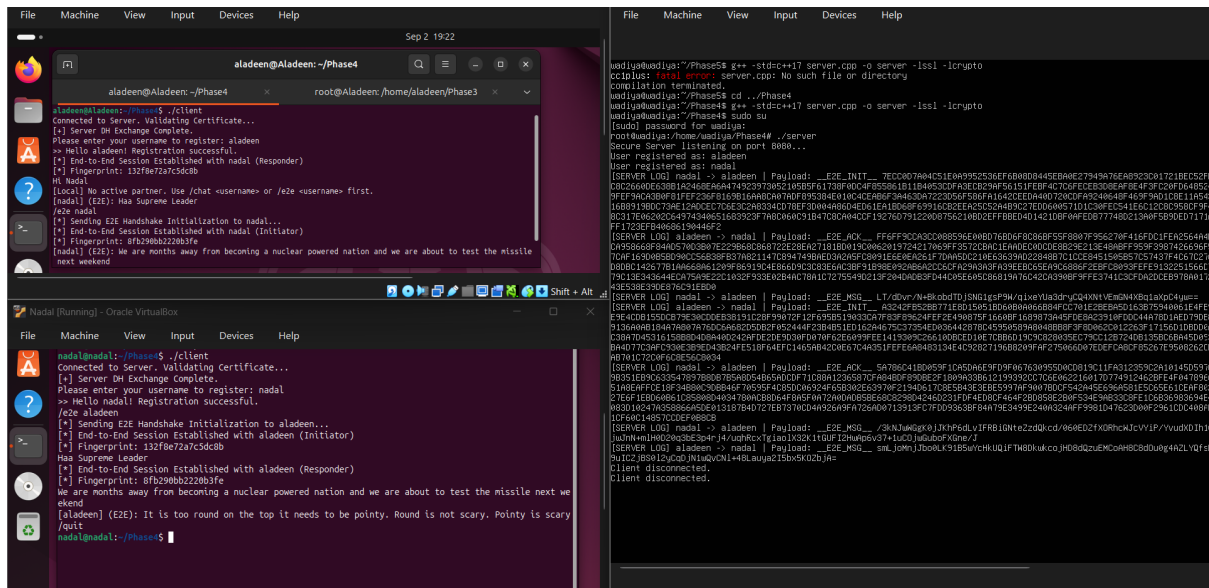


Figure 18: Phase 5 E2E Key Rotation. Client logs display matching, updated key fingerprints and timestamps across multiple rotation cycles, followed by successful post-rotation message decryption.

### 3.5.3 Cryptographic Analysis: The Value of Perfect Forward Secrecy

In Phase 4, the protocol utilized a single static symmetric key for the entirety of the end-to-end session. While this protected the data in transit from the server and passive eavesdroppers, it introduced a severe long-term vulnerability. If an adversary were to passively record all encrypted network traffic over days or weeks and later manage to compromise that single session key (e.g., via a memory leak or client endpoint compromise), they could retroactively decrypt the entire historical conversation.

Perfect Forward Secrecy (PFS) mitigates this catastrophic failure scenario. By automatically rotating the session key every 60 seconds and strictly destroying the old key material from memory, the protocol isolates the impact of a key compromise. If an attacker manages to steal a specific session key, they can *only* decrypt the 60 seconds of traffic protected by that specific key. All prior communications remain cryptographically secure because the past keys have been destroyed and cannot be mathematically derived from the compromised current key. Phase 4 provided confidentiality; Phase 5 ensures that this confidentiality is future-proofed against retroactive decryption.

## 4 Conclusion

This assignment successfully demonstrated the iterative design, implementation, and security analysis of a multi-client chat application. By progressively layering cryptographic

protocols, the project highlighted both the theoretical principles and practical engineering challenges of secure network communication.

The development began with a functional but insecure plaintext baseline (Phase 1), which was subsequently upgraded to utilize Diffie-Hellman key exchanges and AES-256-GCM authenticated encryption for client-server confidentiality (Phase 2). Through practical Man-in-the-Middle (MITM) proxy attacks, the inherent vulnerabilities of anonymous key exchanges were exposed. These vulnerabilities were subsequently addressed by implementing a Public Key Infrastructure (PKI) featuring strict X.509 certificate validation and Proof-of-Possession mechanisms to authenticate the server (Phase 3).

To achieve absolute user privacy and eliminate the central server as a cryptographic vulnerability, the application was further enhanced with an End-to-End (E2E) encryption protocol (Phase 4), successfully reducing the server's role to a zero-knowledge message relay. Finally, the implementation of automated, timer-based key rotation and deterministic collision-avoidance logic established Perfect Forward Secrecy (Phase 5). This final mechanism ensured that the potential compromise of a current session key could not be used to retroactively decrypt historical communications.

Ultimately, the experimental results, network traffic analyses, and MITM vulnerability testing confirm that the objectives of the assignment were fully achieved. The project effectively bridged the gap between raw cryptographic primitives (such as SHA-256, AES-GCM, and RSA) and robust secure protocol design, resulting in a resilient communication architecture.

## 5 References

1. James F. Kurose and Keith W. Ross, Computer Networking: A Top-Down Approach, Pearson, 2012.

## A Source Code

For overall code, here is the link to the GitHub repository along with their respective README files for each Phase: [https://github.com/7-naveen-7/NetworkSecurity\\_Assignment1](https://github.com/7-naveen-7/NetworkSecurity_Assignment1). This part of the report has full code for Phase 1 and for the rest of the Phases, what were added additionally is mentioned

### A.1 Phase 1: Baseline Chat Application

#### A.1.1 Server Implementation

```
1 #include <iostream>
2 #include <map>
3 #include <string>
4 #include <cstring>
5 #include <cstdlib>
6 #include <unistd.h>
7 #include <arpa/inet.h>
8 #include <sys/select.h>
9
10 constexpr int PORT = 8080;
11 constexpr int BUFFER_SIZE = 1024;
12
13 int main() {
14     int server_fd = socket(AF_INET, SOCK_STREAM, 0);
15     struct sockaddr_in address;
16     int opt = 1;
17     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT, &opt, sizeof(opt));
18
19     address.sin_family = AF_INET;
20     address.sin_addr.s_addr = INADDR_ANY;
21     address.sin_port = htons(PORT);
22
23     bind(server_fd, (struct sockaddr *)&address, sizeof(address));
24     listen(server_fd, 5);
25     std::cout << "Wadiya Server listening on port " << PORT << "...\\n";
26     std::cout << "Type '/quit' at any time to gracefully shut down the server.\\n";
27
28     std::map<int, std::string> clients;
29     std::map<int, std::string> client_buffers;
30     fd_set readfds;
31     char buffer[BUFFER_SIZE];
32     bool server_running = true;
33 }
```

```

34 while (server_running) {
35     FD_ZERO(&readfds);
36     FD_SET(STDIN_FILENO, &readfds); // 1. Monitor Server Keyboard
37     FD_SET(server_fd, &readfds);    // 2. Monitor Listening Socket
38     int max_sd = server_fd;
39
40     for (auto const& [sd, username] : clients) {
41         FD_SET(sd, &readfds);
42         if (sd > max_sd) max_sd = sd;
43     }
44
45     if (select(max_sd + 1, &readfds, NULL, NULL, NULL) < 0) continue;
46
47     // EVENT A: Server Administrator Keyboard Input
48     if (FD_ISSET(STDIN_FILENO, &readfds)) {
49         memset(buffer, 0, BUFFER_SIZE);
50         int bytes_read = read(STDIN_FILENO, buffer, BUFFER_SIZE);
51         if (bytes_read > 0) {
52             if (buffer[bytes_read - 1] == '\n') buffer[bytes_read - 1] = '\0';
53             std::string input(buffer);
54
55             if (input == "/quit") {
56                 std::cout << "[Server] Initiating graceful shutdown...\n";
57                 server_running = false; // Break the while loop
58                 continue;
59             } else {
60                 std::cout << "[Server] Unknown command. Type /quit to exit.\n";
61             }
62         }
63     }
64
65     // EVENT B: New Connection
66     if (FD_ISSET(server_fd, &readfds)) {
67         int new_socket = accept(server_fd, NULL, NULL);
68         clients[new_socket] = "";
69         client_buffers[new_socket] = "";
70         const char* welcome = "Welcome! Please enter your username (no spaces):\n";
71
72         ;
73
74         send(new_socket, welcome, strlen(welcome), 0);
75     }
76
77     // EVENT C: Incoming Client Data
78     for (auto it = clients.begin(); it != clients.end(); ) {
79         int sd = it->first;
80
81         if (FD_ISSET(sd, &readfds)) {
82             memset(buffer, 0, BUFFER_SIZE);

```

```

80         int valread = read(sd, buffer, BUFFER_SIZE - 1);
81
82         if (valread <= 0) {
83             std::cout << (it->second.empty() ? "Unknown" : it->second) << "
disconnected.\n";
84             close(sd);
85             client_buffers.erase(sd);
86             it = clients.erase(it);
87             continue;
88         }
89
90         buffer[valread] = '\0';
91         client_buffers[sd] += buffer;
92
93         size_t pos;
94         while ((pos = client_buffers[sd].find('\n')) != std::string::npos) {
95             std::string msg = client_buffers[sd].substr(0, pos);
96             client_buffers[sd].erase(0, pos + 1);
97             if (!msg.empty() && msg.back() == '\r') msg.pop_back();
98
99             std::string sender = it->second.empty() ? "Unregistered" : it->
second;
100             std::cout << "[AUDIT LOG] Relaying plaintext from " << sender << "
: " << msg << "\n";
101
102             if (it->second == "") {
103                 if (msg.find(' ') != std::string::npos) {
104                     std::string err = "[Server] Error: Usernames cannot
contain spaces. Try again.\n";
105                     send(sd, err.c_str(), err.length(), 0);
106                 } else {
107                     it->second = msg;
108                     std::string ack = "Hello " + msg + "! Use /who to see
online users.\n";
109                     send(sd, ack.c_str(), ack.length(), 0);
110                 }
111             } else {
112                 if (msg == "/who") {
113                     std::string who_list = "[Server] Online users: ";
114                     for (auto const& [d_sd, d_name] : clients) {
115                         if (!d_name.empty()) who_list += d_name + ", ";
116                     }
117                     who_list += "\n";
118                     send(sd, who_list.c_str(), who_list.length(), 0);
119                 }
120                 else if (msg.rfind("@", 0) == 0) {
121                     size_t space_pos = msg.find(' ');

```



```

122         if (space_pos != std::string::npos) {
123             std::string target = msg.substr(1, space_pos - 1);
124             std::string actual_msg = msg.substr(space_pos + 1);
125
126             bool found = false;
127             for (auto const& [dest_sd, dest_name] : clients) {
128                 if (dest_name == target) {
129                     std::string f_msg = "[" + sender + "]: " +
actual_msg + "\n";
130                     send(dest_sd, f_msg.c_str(), f_msg.length(),
0);
131                     found = true;
132                     break;
133                 }
134             }
135             if (!found) {
136                 std::string err = "[Server] User '" + target + "'
offline.\n";
137                 send(sd, err.c_str(), err.length(), 0);
138             }
139         }
140     }
141 }
142 }
143 }
144 ++it;
145 }
146 }
147
148 // --- GRACEFUL SHUTDOWN SEQUENCE ---
149 std::cout << "Disconnecting all active clients...\n";
150 for (auto const& [sd, username] : clients) {
151     std::string quit_msg = "[Server] Server is shutting down. Goodbye!\n";
152     send(sd, quit_msg.c_str(), quit_msg.length(), 0);
153     close(sd);
154 }
155
156 close(server_fd);
157 std::cout << "Wadiya Server successfully terminated.\n";
158 return 0;
159 }

```

Listing 1: Phase 1 Server Implementation (server.cpp)

### A.1.2 Client Implementation

```

1  #include <iostream>
2  #include <string>
3  #include <cstring>
4  #include <cstdlib>
5  #include <unistd.h>
6  #include <arpa/inet.h>
7  #include <sys/select.h>
8
9  constexpr int PORT = 8080;
10 constexpr int BUFFER_SIZE = 1024;
11 constexpr const char* SERVER_IP = "10.0.2.10";
12
13 int main() {
14     int sock = socket(AF_INET, SOCK_STREAM, 0);
15     struct sockaddr_in serv_addr;
16     serv_addr.sin_family = AF_INET;
17     serv_addr.sin_port = htons(PORT);
18     inet_pton(AF_INET, SERVER_IP, &serv_addr.sin_addr);
19
20     if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
21         std::cerr << "Connection Failed.\n";
22         return -1;
23     }
24
25     fd_set readfds;
26     char buffer[BUFFER_SIZE];
27
28     bool registered = false;
29     std::string active_partner = "";
30     std::string rx_buffer = ""; // For receiving framed messages
31
32     while (true) {
33         FD_ZERO(&readfds);
34         FD_SET(STDIN_FILENO, &readfds);
35         FD_SET(sock, &readfds);
36
37         if (select(sock + 1, &readfds, NULL, NULL, NULL) < 0) break;
38
39         // EVENT A: User typed on keyboard
40         if (FD_ISSET(STDIN_FILENO, &readfds)) {
41             memset(buffer, 0, BUFFER_SIZE);
42             int bytes_read = read(STDIN_FILENO, buffer, BUFFER_SIZE);
43             if (bytes_read > 0) {
44                 if (buffer[bytes_read - 1] == '\n') buffer[bytes_read - 1] = '\0';
45                 std::string input(buffer);
46                 std::string to_send = "";

```

```

47
48     if (!registered) {
49         to_send = input + "\n"; // Append frame delimiter
50         registered = true;
51     } else {
52         if (input.rfind("@", 0) == 0) {
53             size_t space_pos = input.find(' ');
54             if (space_pos != std::string::npos) {
55                 active_partner = input.substr(1, space_pos - 1);
56                 to_send = input + "\n";
57             } else std::cout << "[Local] Use @username message\n";
58         }
59         else if (input.rfind("/chat ", 0) == 0) {
60             active_partner = input.substr(6);
61             std::cout << "[Local] Active chat partner set to: " <<
active_partner << "\n";
62         }
63         else if (input == "/who" || input.rfind("/e2e ", 0) == 0) {
64             to_send = input + "\n";
65         }
66         else if (input == "/quit") {
67             std::cout << "[Local] Disconnecting...\n";
68             break;
69         }
70         else {
71             if (active_partner.empty()) {
72                 std::cout << "[Local] No active partner. Use /chat <
username> first.\n";
73             } else {
74                 to_send = "@" + active_partner + " " + input + "\n";
75             }
76         }
77     }
78     if (!to_send.empty()) {
79         send(sock, to_send.c_str(), to_send.length(), 0);
80     }
81 }
82
83
84 // EVENT B: Incoming message from Server
85 if (FD_ISSET(sock, &readfds)) {
86     memset(buffer, 0, BUFFER_SIZE);
87     int valread = read(sock, buffer, BUFFER_SIZE - 1);
88     if (valread <= 0) {
89         std::cout << "\nServer disconnected.\n";
90         break;
91     } else {

```

```

92         buffer[valread] = '\0';
93         rx_buffer += buffer;
94         size_t pos;
95         // Process full frames only
96         while ((pos = rx_buffer.find('\n')) != std::string::npos) {
97             std::string msg = rx_buffer.substr(0, pos);
98             rx_buffer.erase(0, pos + 1);
99             std::cout << ">> " << msg << "\n";
100         }
101     }
102 }
103 }
104 close(sock);
105 return 0;
106 }

```

Listing 2: Phase 1 Client Implementation (client.cpp)

## A.2 Phase 2: Client-Server Confidentiality

```

1  // The following header files were added: <openssl/bn.h>, <openssl/sha.h>, <openssl/
   evp.h>, <openssl/rand.h>
2
3  // 1. RFC 3526 MODP 3072-bit Prime number
4  const char* MODP_3072_PRIME =
5      "FFFFFFFFFFFFFFFFC90FDAA22168C234C4C6628B80DC1CD1"
6      "29024E088A67CC74020BBEA63B139B22514A08798E3404DD"
7      "EF9519B3CD3A431B302B0A6DF25F14374FE1356D6D51C245"
8      "E485B576625E7EC6F44C42E9A637ED6B0BFF5CB6F406B7ED"
9      "EE386BFB5A899FA5AE9F24117C4B1FE649286651ECE45B3D"
10     "C2007CB8A163BF0598DA48361C55D39A69163FA8FD24CF5F"
11     "83655D23DCA3AD961C62F356208552BB9ED529077096966D"
12     "670C354E4ABC9804F1746C08CA18217C32905E462E36CE3B"
13     "E39E772C180E86039B2783A2EC07A28FB5C55DF06F4C52C9"
14     "DE2BCBF6955817183995497CEA956AE515D2261898FA0510"
15     "15728E5A8AAAC42DAD33170D04507A33A85521ABDF1CBA64"
16     "ECFB850458DBEF0A8AEA71575D060C7DB3970F85A6E1E4C7"
17     "ABF5AE8CDB0933D71E8C94E04A25619DCEE3D2261AD2EE6B"
18     "F12FFA06D98A0864D87602733EC86A64521F2B18177B200C"
19     "BBE117577A615D6C770988C0BAD946E208E24FA074E5AB31"
20     "43DB5BFCE0FD108E4B82D120A93AD2CAFFFFFFFFFFFFFFFF";
21
22 // 2. Cryptographic & Utility Functions (Both Client and Server)
23 bool send_all(int sock, const std::string& data) {
24     size_t total_sent = 0;

```

```

25     while (total_sent < data.size()) {
26         ssize_t n = send(sock, data.data() + total_sent, data.size() - total_sent, 0);
27         if (n <= 0) {
28             if (n < 0 && errno == EINTR) continue;
29             return false;
30         }
31         total_sent += (size_t)n;
32     }
33     return true;
34 }
35
36 std::string b64_encode(const std::string& in) {
37     std::string out(4 * ((in.size() + 2) / 3), '\\0');
38     int len = EVP_EncodeBlock((unsigned char*)out.data(), (const unsigned char*)in.
39     data(), in.size());
40     out.resize(len);
41     return out;
42 }
43
44 std::string b64_decode(const std::string& in) {
45     int pad = 0;
46     if (in.length() > 0 && in[in.length() - 1] == '=') pad++;
47     if (in.length() > 1 && in[in.length() - 2] == '=') pad++;
48     std::string out(3 * in.length() / 4, '\\0');
49     int len = EVP_DecodeBlock((unsigned char*)out.data(), (const unsigned char*)in.
50     data(), in.length());
51     out.resize(len - pad);
52     return out;
53 }
54
55 std::string get_fingerprint(const unsigned char* aes_key) {
56     unsigned char hash[SHA256_DIGEST_LENGTH];
57     SHA256(aes_key, 32, hash);
58     char hex[17];
59     for (int i = 0; i < 8; i++) sprintf(hex + (i * 2), "%02x", hash[i]);
60     hex[16] = '\\0';
61     return std::string(hex);
62 }
63
64 std::string aes_encrypt(const std::string& plaintext, const unsigned char* key) {
65     EVP_CIPHER_CTX* ctx = EVP_CIPHER_CTX_new();
66     unsigned char iv[12];
67     RAND_bytes(iv, sizeof(iv));
68
69     std::string ciphertext;
70     ciphertext.resize(plaintext.size() + EVP_MAX_BLOCK_LENGTH);
71     int len = 0, ciphertext_len = 0;

```

```

70
71     EVP_EncryptInit_ex(ctx, EVP_aes_256_gcm(), NULL, key, iv);
72     EVP_EncryptUpdate(ctx, (unsigned char*)&ciphertext[0], &len, (unsigned char*)
plaintext.c_str(), plaintext.size());
73     ciphertext_len = len;
74     EVP_EncryptFinal_ex(ctx, (unsigned char*)&ciphertext[0] + len, &len);
75     ciphertext_len += len;
76     ciphertext.resize(ciphertext_len);
77
78     unsigned char tag[16];
79     EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_GET_TAG, 16, tag);
80     EVP_CIPHER_CTX_free(ctx);
81
82     std::string bin_result = "";
83     bin_result.append((char*)iv, 12);
84     bin_result.append((char*)tag, 16);
85     bin_result.append(ciphertext);
86     return b64_encode(bin_result);
87 }
88
89 bool aes_decrypt(const std::string& b64_input, const unsigned char* key, std::string&
decrypted_out) {
90     std::string input = b64_decode(b64_input);
91     if (input.size() < 28) return false;
92     const unsigned char* iv = (const unsigned char*)input.data();
93     const unsigned char* tag = (const unsigned char*)input.data() + 12;
94     const unsigned char* ciphertext = (const unsigned char*)input.data() + 28;
95     int ciphertext_len = input.size() - 28;
96
97     EVP_CIPHER_CTX* ctx = EVP_CIPHER_CTX_new();
98     EVP_DecryptInit_ex(ctx, EVP_aes_256_gcm(), NULL, key, iv);
99
100     std::string plaintext;
101     plaintext.resize(ciphertext_len);
102     int len = 0, plaintext_len = 0;
103
104     EVP_DecryptUpdate(ctx, (unsigned char*)&plaintext[0], &len, ciphertext,
ciphertext_len);
105     plaintext_len = len;
106     EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_GCM_SET_TAG, 16, (void*)tag);
107
108     int ret = EVP_DecryptFinal_ex(ctx, (unsigned char*)&plaintext[0] + len, &len);
109     EVP_CIPHER_CTX_free(ctx);
110
111     if (ret > 0) {
112         plaintext_len += len;
113         plaintext.resize(plaintext_len);

```

```

114         decrypted_out = plaintext;
115         return true;
116     }
117     return false;
118 }
119
120 // 3. State Management (added to Server)
121 struct ClientSession {
122     std::string username = "";
123     unsigned char aes_key[32];
124     std::string rx_buffer = "";
125     bool dh_completed = false;
126 };
127
128 // Replaced std::map<int, std::string> clients; with:
129 std::map<int, ClientSession> clients;
130
131 // 4. Diffie-Hellman Key Exchange (Server)
132 if (!it->second.dh_completed) {
133     BN_CTX* ctx = BN_CTX_new();
134     BIGNUM* p = NULL; BIGNUM* g = NULL;
135     BIGNUM* priv_b = BN_new();
136     BIGNUM* pub_b = BN_new();
137     BIGNUM* client_pub = NULL;
138     BIGNUM* shared_secret = BN_new();
139
140     BN_hex2bn(&p, MODP_3072_PRIME);
141     BN_hex2bn(&g, "2");
142     BN_hex2bn(&client_pub, frame.c_str());
143
144     BN_rand_range(priv_b, p);
145     BN_mod_exp(pub_b, g, priv_b, p, ctx);
146     BN_mod_exp(shared_secret, client_pub, priv_b, p, ctx); //  $S = A^b \bmod p$ 
147
148     char* pub_b_hex = BN_bn2hex(pub_b);
149     std::string server_pub_msg = std::string(pub_b_hex) + "\n";
150     if (!send_all(sd, server_pub_msg)) {
151         std::cout << "[Session " << sd << "] Failed to send DH public value.\n";
152     }
153
154     int num_bytes = BN_num_bytes(shared_secret);
155     unsigned char* secret_bytes = new unsigned char[num_bytes];
156     BN_bn2bin(shared_secret, secret_bytes);
157     SHA256(secret_bytes, num_bytes, it->second.aes_key); // Key Derivation
158
159     it->second.dh_completed = true;

```



```

160     std::cout << "[Session " << sd << "] DH Exchange Complete. Fingerprint: " <<
        get_fingerprint(it->second.aes_key) << "\n";
161
162     delete[] secret_bytes;
163     OPENSSL_free(pub_b_hex);
164     BN_free(p); BN_free(g); BN_free(priv_b); BN_free(pub_b); BN_free(client_pub);
        BN_free(shared_secret); BN_CTX_free(ctx);
165 }
166
167 // 5. Diffie-Hellman Key Exchange (Client)
168 BN_CTX* ctx = BN_CTX_new();
169 BIGNUM* p = NULL; BIGNUM* g = NULL;
170 BIGNUM* priv_a = BN_new();
171 BIGNUM* pub_a = BN_new();
172
173 BN_hex2bn(&p, MODP_3072_PRIME);
174 BN_hex2bn(&g, "2");
175 BN_rand_range(priv_a, p);
176 BN_mod_exp(pub_a, g, priv_a, p, ctx);
177
178 char* pub_a_hex = BN_bn2hex(pub_a);
179 std::string init_msg = std::string(pub_a_hex) + "\n";
180 if (!send_all(sock, init_msg)) {
181     std::cerr << "Failed to send DH public value.\n";
182     return -1;
183 }
184
185 std::string server_pub_hex = "";
186 char c;
187 while (read(sock, &c, 1) > 0) {
188     if (c == '\n') break;
189     server_pub_hex += c;
190 }
191
192 BIGNUM* server_pub = NULL;
193 BIGNUM* shared_secret = BN_new();
194 BN_hex2bn(&server_pub, server_pub_hex.c_str());
195 BN_mod_exp(shared_secret, server_pub, priv_a, p, ctx);
196
197 unsigned char aes_key[32];
198 int num_bytes = BN_num_bytes(shared_secret);
199 unsigned char* secret_bytes = new unsigned char[num_bytes];
200 BN_bn2bin(shared_secret, secret_bytes);
201 SHA256(secret_bytes, num_bytes, aes_key);
202
203 delete[] secret_bytes;
204 OPENSSL_free(pub_a_hex);

```

```

205 BN_free(p); BN_free(g); BN_free(priv_a); BN_free(pub_a); BN_free(server_pub); BN_free(
    shared_secret); BN_CTX_free(ctx);
206
207 std::cout << "DH Key Exchange Complete. Fingerprint: " << get_fingerprint(aes_key) <<
    "\n";
208
209 // 6. Code to prove that AES-GCM detects tampering (Client)
210 bool trigger_tampering = false;
211
212 // Inside FD_ISSET(STDIN_FILENO, &readfds)
213 if (input == "/tamper") {
214     trigger_tampering = true;
215     std::cout << "[!] TAMPER MODE ARMED: The next incoming message will have one byte
        altered.\n";
216     continue;
217 }
218
219 // Inside FD_ISSET(sock, &readfds) during frame parsing
220 if (trigger_tampering) {
221     std::cout << "\n--- INTERCEPTED INCOMING PACKET ---\n";
222     std::cout << "Original Base64 Payload: " << frame << "\n";
223
224     if (frame.length() > 10) {
225         frame[10] = (frame[10] == 'A') ? 'B' : 'A';
226     }
227
228     std::cout << "Tampered Base64 Payload: " << frame << "\n";
229     std::cout << "Attempting AES-GCM Decryption...\n";
230     trigger_tampering = false;
231 }

```

Listing 3: Phase 2 Core Cryptographic Implementations

### A.3 Phase 3: Public Key Infrastructure (PKI)

```

1 // server.cpp added with following header files: <fstream>, <openssl/pem.h>, <openssl/
    x509.h>, <openssl/err.h>
2 // client.cpp added with following header files: <openssl/pem.h>, <openssl/x509.h>, <
    openssl/x509_vfy.h>
3
4 // Path of CA Cert is, CA_CERT_PATH = "ca_cert.pem"
5
6 // 1. Utility Functions & State Management (Server)
7 std::string read_file(const std::string& filepath) {
8     std::ifstream t(filepath);

```

```

9     return std::string((std::istreambuf_iterator<char>(t)), std::istreambuf_iterator<
    char>());
10 }
11
12 enum class AuthState { WAIT_CHALLENGE, WAIT_DH, ENCRYPTED };
13
14 // Modified ClientSession struct
15 struct ClientSession {
16     std::string username = "";
17     unsigned char aes_key[32];
18     std::string rx_buffer = "";
19     AuthState state = AuthState::WAIT_CHALLENGE; // Replaced bool dh_completed
20 };
21
22 // 3. PKI Initialization (Server main function)
23 std::string server_cert_str = read_file("server_cert.pem");
24 if (server_cert_str.empty()) {
25     std::cerr << "Failed to read server_cert.pem\n";
26     return -1;
27 }
28
29 FILE* key_file = fopen("server_key.pem", "r");
30 if (!key_file) {
31     std::cerr << "Failed to open server_key.pem\n";
32     return -1;
33 }
34 EVP_PKEY* server_pkey = PEM_read_PrivateKey(key_file, NULL, NULL, NULL);
35 fclose(key_file);
36 if (!server_pkey) {
37     std::cerr << "Failed to load private key\n";
38     return -1;
39 }
40
41 // at end of main():
42 EVP_PKEY_free(server_pkey);
43
44 // 4. Added Certificate Transmission Inside FD_ISSET(server_fd, &readfds)
45 send_all(new_socket, server_cert_str);
46
47 // 5. Proof of Possession Signing Logic (Server message processing)
48 if (it->second.state == AuthState::WAIT_CHALLENGE) {
49     std::string challenge = b64_decode(frame);
50
51     // Sign the challenge using the server's private key
52     EVP_MD_CTX* mdctx = EVP_MD_CTX_new();
53     EVP_DigestSignInit(mdctx, NULL, EVP_sha256(), NULL, server_pkey);
54     EVP_DigestSignUpdate(mdctx, challenge.data(), challenge.size());

```

```

55
56     size_t sig_len;
57     EVP_DigestSignFinal(mdctx, NULL, &sig_len);
58     unsigned char* sig = (unsigned char*)malloc(sig_len);
59     EVP_DigestSignFinal(mdctx, sig, &sig_len);
60     EVP_MD_CTX_free(mdctx);
61
62     std::string sig_str((char*)sig, sig_len);
63     free(sig);
64
65     // Send the signature back to the client
66     std::string response = b64_encode(sig_str) + "\n";
67     send_all(sd, response);
68
69     it->second.state = AuthState::WAIT_DH;
70     std::cout << "[Session " << sd << "] Signed challenge. Awaiting DH Key...\n";
71 }
72
73 // 6. Certificate Validation & Proof of Possession Verification (Client main function)
74 std::string cert_data = "";
75 char c;
76 const std::string cert_end_marker = "-----END CERTIFICATE-----\n";
77 while (read(sock, &c, 1) > 0) {
78     cert_data += c;
79     if (cert_data.length() >= cert_end_marker.length()) {
80         if (cert_data.substr(cert_data.length() - cert_end_marker.length()) ==
81             cert_end_marker) {
82             break;
83         }
84     }
85 }
86
87 // Load received cert into X509 structure
88 BIO* bio = BIO_new_mem_buf(cert_data.data(), cert_data.size());
89 X509* server_cert = PEM_read_bio_X509(bio, NULL, NULL, NULL);
90 BIO_free(bio);
91
92 if (!server_cert) {
93     std::cerr << "[!] CRITICAL: Failed to parse server certificate. Aborting.\n";
94     close(sock); return -1;
95 }
96
97 // Step A & B: Validate Signature and Validity Period using X509_STORE
98 X509_STORE* store = X509_STORE_new();
99 if (X509_STORE_load_locations(store, CA_CERT_PATH, NULL) != 1) {
100     std::cerr << "[!] CRITICAL: Failed to load Trusted CA root certificate. Aborting.\n";

```

```

100     close(sock); return -1;
101 }
102
103 X509_STORE_CTX* vrfy_ctx = X509_STORE_CTX_new();
104 X509_STORE_CTX_init(vrfy_ctx, store, server_cert, NULL);
105 int verify_result = X509_verify_cert(vrfy_ctx);
106 X509_STORE_CTX_free(vrfy_ctx);
107 X509_STORE_free(store);
108
109 if (verify_result != 1) {
110     std::cerr << "[!] CRITICAL: Certificate Validation Failed! (Untrusted CA or
111     Expired). Aborting.\n";
112     close(sock); return -1;
113 }
114
115 // Step C: Validate Common Name (CN) matches EXPECTED Server IP
116 X509_NAME* subj = X509_get_subject_name(server_cert);
117 char cn_buf[256];
118 X509_NAME_get_text_by_NID(subj, NID_commonName, cn_buf, sizeof(cn_buf));
119 if (std::string(cn_buf) != SERVER_IP) {
120     std::cerr << "[!] CRITICAL: Certificate CN mismatch! Expected: " << SERVER_IP << "
121     , Found: " << cn_buf << ". Aborting.\n";
122     close(sock); return -1;
123 }
124
125 // Proof of Possession (Challenge-Response)
126 unsigned char challenge[32];
127 RAND_bytes(challenge, sizeof(challenge));
128 std::string challenge_str((char*)challenge, sizeof(challenge));
129
130 // Send challenge to server
131 send_all(sock, b64_encode(challenge_str) + "\n");
132
133 // Read the signature back
134 std::string sig_b64 = "";
135 while (read(sock, &c, 1) > 0) {
136     if (c == '\n') break;
137     sig_b64 += c;
138 }
139
140 std::string sig_raw = b64_decode(sig_b64);
141
142 // Verify signature using the Public Key in the validated certificate
143 EVP_PKEY* server_pubkey = X509_get_pubkey(server_cert);
144 EVP_MD_CTX* mdctx = EVP_MD_CTX_new();
145 EVP_DigestVerifyInit(mdctx, NULL, EVP_sha256(), NULL, server_pubkey);
146 EVP_DigestVerifyUpdate(mdctx, challenge, sizeof(challenge));

```

```

144 int sig_valid = EVP_DigestVerifyFinal(mdctx, (const unsigned char*)sig_raw.data(),
    sig_raw.size());
145
146 EVP_MD_CTX_free(mdctx);
147 EVP_PKEY_free(server_pubkey);
148 X509_free(server_cert);
149
150 if (sig_valid != 1) {
151     std::cerr << "[!] CRITICAL: Proof of Possession Failed! Server doesn't hold the
    private key. Aborting.\n";
152     close(sock); return -1;
153 }

```

Listing 4: Phase 3 PKI, Certificates, and Proof of Possession

## A.4 Phase 4: End-to-End Encryption

```

1  // --- State structures for Phase 4 ---
2  struct E2ESession {
3      bool established = false;
4      BIGNUM* priv_key = nullptr; // Held temporarily during handshake
5      unsigned char aes_key[32];
6  };
7
8  // Map tracking for peers
9  std::map<std::string, E2ESession> e2e_sessions; // Keeps track of peers
10
11 // E2E command description
12 if (input.rfind("/e2e ", 0) == 0) {
13     active_partner = input.substr(5);
14
15     // Generate E2E DH Keys
16     BN_CTX* ctx = BN_CTX_new();
17     BIGNUM* modp = NULL; BIGNUM* gen = NULL;
18     BN_hex2bn(&modp, MODP_3072_PRIME); BN_hex2bn(&gen, "2");
19
20     BIGNUM* e2e_priv = BN_new();
21     BIGNUM* e2e_pub = BN_new();
22     BN_rand_range(e2e_priv, modp);
23     BN_mod_exp(e2e_pub, gen, e2e_priv, modp, ctx);
24
25     // Save private key for when ACK comes back
26     if (e2e_sessions[active_partner].priv_key) BN_free(e2e_sessions[active_partner].
    priv_key);
27     e2e_sessions[active_partner].priv_key = e2e_priv;

```

```

28     e2e_sessions[active_partner].established = false;
29
30     char* e2e_pub_hex = BN_bn2hex(e2e_pub);
31     to_send = "@" + active_partner + " __E2E_INIT__ " + e2e_pub_hex;
32
33     std::cout << "[*] Sending E2E Handshake Initialization to " << active_partner << "
34     ...\\n";
35
36     OPENSSL_free(e2e_pub_hex); BN_free(modp); BN_free(gen); BN_free(e2e_pub);
37     BN_CTX_free(ctx);
38 }
39
40 // Encrypt payload first with E2E key if established
41 if (e2e_sessions[active_partner].established) {
42     std::string e2e_cipher = aes_encrypt(input, e2e_sessions[active_partner].aes_key);
43     to_send = "@" + active_partner + " __E2E_MSG__ " + e2e_cipher;
44 } else {
45     to_send = "@" + active_partner + " " + input;
46 }
47
48 // INTERCEPT: Target initiated an E2E exchange
49 if (body.rfind("__E2E_INIT__ ", 0) == 0) {
50     std::string peer_pub_hex = body.substr(13);
51
52     BN_CTX* ctx = BN_CTX_new();
53     BIGNUM* modp = NULL; BIGNUM* gen = NULL;
54     BN_hex2bn(&modp, MODP_3072_PRIME); BN_hex2bn(&gen, "2");
55
56     BIGNUM* peer_pub = NULL;
57     BN_hex2bn(&peer_pub, peer_pub_hex.c_str());
58
59     BIGNUM* e2e_priv = BN_new(); BIGNUM* e2e_pub = BN_new();
60     BN_rand_range(e2e_priv, modp);
61     BN_mod_exp(e2e_pub, gen, e2e_priv, modp, ctx);
62
63     BIGNUM* shared_secret = BN_new();
64     BN_mod_exp(shared_secret, peer_pub, e2e_priv, modp, ctx);
65
66     int n_bytes = BN_num_bytes(shared_secret);
67     unsigned char* s_bytes = new unsigned char[n_bytes];
68     BN_bn2bin(shared_secret, s_bytes);
69     SHA256(s_bytes, n_bytes, e2e_sessions[sender].aes_key);
70     e2e_sessions[sender].established = true;
71
72     // Send ACK back to initiator
73     char* e2e_pub_hex = BN_bn2hex(e2e_pub);
74     std::string ack_msg = "@" + sender + " __E2E_ACK__ " + e2e_pub_hex;

```



```

73     send_all(sock, aes_encrypt(ack_msg, client_server_aes_key) + "\n");
74
75     std::cout << "[*] End-to-End Session Established with " << sender << " (Responder)
76     \n";
77     std::cout << "[*] Fingerprint: " << get_fingerprint(e2e_sessions[sender].aes_key)
78     << "\n";
79
80     delete[] s_bytes; OPENSSL_free(e2e_pub_hex);
81     BN_free(modp); BN_free(gen); BN_free(peer_pub); BN_free(e2e_priv); BN_free(e2e_pub
82     ); BN_free(shared_secret); BN_CTX_free(ctx);
83 }
84 // INTERCEPT: Target acknowledged our E2E request
85 else if (body.rfind("__E2E_ACK__ ", 0) == 0) {
86     std::string peer_pub_hex = body.substr(12);
87
88     BN_CTX* ctx = BN_CTX_new();
89     BIGNUM* modp = NULL;
90     BN_hex2bn(&modp, MODP_3072_PRIME);
91
92     BIGNUM* peer_pub = NULL;
93     BN_hex2bn(&peer_pub, peer_pub_hex.c_str());
94
95     BIGNUM* shared_secret = BN_new();
96     BN_mod_exp(shared_secret, peer_pub, e2e_sessions[sender].priv_key, modp, ctx);
97
98     int n_bytes = BN_num_bytes(shared_secret);
99     unsigned char* s_bytes = new unsigned char[n_bytes];
100     BN_bn2bin(shared_secret, s_bytes);
101     SHA256(s_bytes, n_bytes, e2e_sessions[sender].aes_key);
102
103     e2e_sessions[sender].established = true;
104     BN_free(e2e_sessions[sender].priv_key);
105     e2e_sessions[sender].priv_key = nullptr; // Clean up
106
107     std::cout << "[*] End-to-End Session Established with " << sender << " (Initiator)
108     \n";
109     std::cout << "[*] Fingerprint: " << get_fingerprint(e2e_sessions[sender].aes_key)
110     << "\n";
111
112     delete[] s_bytes; BN_free(modp); BN_free(peer_pub); BN_free(shared_secret);
113     BN_CTX_free(ctx);
114 }
115 // INTERCEPT: Double-encrypted E2E Message
116 else if (body.rfind("__E2E_MSG__ ", 0) == 0) {
117     std::string b64_cipher = body.substr(12);
118     std::string plaintext;

```

```

113     if (e2e_sessions[sender].established && aes_decrypt(b64_cipher, e2e_sessions[
114         sender].aes_key, plaintext)) {
115         std::cout << "[" << sender << "] (E2E): " << plaintext << "\n";
116     } else {
117         std::cout << "[ERROR] Failed to decrypt E2E message from " << sender << ". Key
118         mismatch.\n";
119     }
120 }
121
122 // INSTRUMENTATION: Log the routed payload payload (server)
123 std::cout << "[SERVER LOG] " << it->second.username
124     << " -> " << target << " | Payload: " << body << "\n";

```

Listing 5: Phase 4 End-to-End Encryption (E2E) Logic

## A.5 Phase 5: Perfect Forward Secrecy

```

1 // Forward Secrecy state
2 // No changes were made to server. Changes were only made to client.
3
4 struct E2ESession {
5     bool established = false;
6     BIGNUM* priv_key = nullptr; // Current phase handshakes
7     BIGNUM* next_priv_key = nullptr; // For rekeying
8     unsigned char aes_key[32]; // Active Key
9     unsigned char old_aes_key[32]; // Previous Key (for in-flight msgs during
10     rotation)
11     bool has_old_key = false;
12
13     std::chrono::steady_clock::time_point last_rekey_time;
14     enum class RekeyState { NONE, INITIATED } rekey_state = RekeyState::NONE;
15 };
16
17 // timestamp utility (for client)
18 std::string get_timestamp() {
19     auto t = std::time(nullptr);
20     auto tm = *std::localtime(&t);
21     char buf[64];
22     std::strftime(buf, sizeof(buf), "%H:%M:%S", &tm);
23     return std::string(buf);
24 }
25
26 // Phase 5 Timer: Select timeout of 1 second to routinely check rotation clock
27 struct timeval tv;
28 tv.tv_sec = 1;

```

```

28 tv.tv_usec = 0;
29
30 int activity = select(sock + 1, &readfds, NULL, NULL, &tv);
31 if (activity < 0) break;
32
33 // 60-Second Auto-Rekeying
34 auto now = std::chrono::steady_clock::now();
35 for (auto& [peer, session] : e2e_sessions) {
36     if (session.established && session.rekey_state == E2ESession::RekeyState::NONE) {
37         if (std::chrono::duration_cast<std::chrono::seconds>(now - session.
38             last_rekey_time).count() >= 60) {
39             BN_CTX* ctx = BN_CTX_new();
40             BIGNUM* modp = NULL; BIGNUM* gen = NULL;
41             BN_hex2bn(&modp, MODP_3072_PRIME); BN_hex2bn(&gen, "2");
42
43             BIGNUM* e2e_priv = BN_new(); BIGNUM* e2e_pub = BN_new();
44             BN_rand_range(e2e_priv, modp); BN_mod_exp(e2e_pub, gen, e2e_priv, modp,
45                 ctx);
46
47             if (session.next_priv_key) BN_free(session.next_priv_key);
48             session.next_priv_key = e2e_priv;
49             session.rekey_state = E2ESession::RekeyState::INITIATED;
50
51             char* e2e_pub_hex = BN_bn2hex(e2e_pub);
52             std::string to_send = "@" + peer + " __E2E_REKEY_INIT__ " + e2e_pub_hex;
53             send_all(sock, aes_encrypt(to_send, client_server_aes_key) + "\n");
54
55             OPENSSL_free(e2e_pub_hex); BN_free(modp); BN_free(gen); BN_free(e2e_pub);
56             BN_CTX_free(ctx);
57         }
58     }
59 }
60
61 // Handling Rekey Initialization
62 else if (body.rfind("__E2E_REKEY_INIT__ ", 0) == 0) {
63     if (e2e_sessions[sender].rekey_state == E2ESession::RekeyState::INITIATED) {
64         // Glare Detected! Tie-breaker: Lexicographical username comparison
65         if (my_username < sender) {
66             // I am dictator. Ignore peer's INIT.
67             continue;
68         } else {
69             // I yield. Clear my pending state.
70             if (e2e_sessions[sender].next_priv_key) BN_free(e2e_sessions[sender].
71                 next_priv_key);
72             e2e_sessions[sender].next_priv_key = nullptr;
73             e2e_sessions[sender].rekey_state = E2ESession::RekeyState::NONE;
74         }
75     }
76 }

```

```

71     }
72
73     std::string peer_pub_hex = body.substr(19);
74     BN_CTX* ctx = BN_CTX_new();
75     BIGNUM* modp = NULL; BIGNUM* gen = NULL;
76     BN_hex2bn(&modp, MODP_3072_PRIME); BN_hex2bn(&gen, "2");
77     BIGNUM* peer_pub = NULL; BN_hex2bn(&peer_pub, peer_pub_hex.c_str());
78     BIGNUM* e2e_priv = BN_new(); BIGNUM* e2e_pub = BN_new(); BIGNUM* shared_secret =
BN_new();
79
80     BN_rand_range(e2e_priv, modp); BN_mod_exp(e2e_pub, gen, e2e_priv, modp, ctx);
81     BN_mod_exp(shared_secret, peer_pub, e2e_priv, modp, ctx);
82
83     // Backup old key
84     memcpy(e2e_sessions[sender].old_aes_key, e2e_sessions[sender].aes_key, 32);
85     e2e_sessions[sender].has_old_key = true;
86
87     // Install new key
88     int n_bytes = BN_num_bytes(shared_secret); unsigned char* s_bytes = new unsigned
char[n_bytes];
89     BN_bn2bin(shared_secret, s_bytes);
90     SHA256(s_bytes, n_bytes, e2e_sessions[sender].aes_key);
91     e2e_sessions[sender].last_rekey_time = std::chrono::steady_clock::now();
92
93     // Send ACK
94     char* e2e_pub_hex = BN_bn2hex(e2e_pub);
95     std::string ack_msg = "@" + sender + " __E2E_REKEY_ACK__ " + e2e_pub_hex;
96     send_all(sock, aes_encrypt(ack_msg, client_server_aes_key) + "\n");
97
98     std::cout << "[" << get_timestamp() << "] [REKEY] Key rotated (Responder) with "
<< sender << "\n";
99     std::cout << "[" << get_timestamp() << "] [REKEY] New Fingerprint: " <<
get_fingerprint(e2e_sessions[sender].aes_key) << "\n";
100
101     delete[] s_bytes; OPENSSL_free(e2e_pub_hex); BN_free(modp); BN_free(gen); BN_free(
peer_pub); BN_free(e2e_priv); BN_free(e2e_pub); BN_free(shared_secret);
BN_CTX_free(ctx);
102 }
103
104 // Handle Rekey Ack
105 else if (body.rfind("__E2E_REKEY_ACK__ ", 0) == 0) {
106     std::string peer_pub_hex = body.substr(18);
107     BN_CTX* ctx = BN_CTX_new();
108     BIGNUM* modp = NULL; BN_hex2bn(&modp, MODP_3072_PRIME);
109     BIGNUM* peer_pub = NULL; BN_hex2bn(&peer_pub, peer_pub_hex.c_str());
110     BIGNUM* shared_secret = BN_new();

```

```

111 BN_mod_exp(shared_secret, peer_pub, e2e_sessions[sender].next_priv_key, modp, ctx)
112 ;
113 memcpy(e2e_sessions[sender].old_aes_key, e2e_sessions[sender].aes_key, 32);
114 e2e_sessions[sender].has_old_key = true;
115
116 int n_bytes = BN_num_bytes(shared_secret); unsigned char* s_bytes = new unsigned
117 char[n_bytes];
118 BN_bn2bin(shared_secret, s_bytes);
119 SHA256(s_bytes, n_bytes, e2e_sessions[sender].aes_key);
120 e2e_sessions[sender].last_rekey_time = std::chrono::steady_clock::now();
121
122 BN_free(e2e_sessions[sender].next_priv_key); e2e_sessions[sender].next_priv_key =
123 nullptr;
124 e2e_sessions[sender].rekey_state = E2ESession::RekeyState::NONE;
125
126 std::cout << "[" << get_timestamp() << "] [REKEY] Key rotated (Initiator) with "
127 << sender << "\n";
128 std::cout << "[" << get_timestamp() << "] [REKEY] New Fingerprint: " <<
129 get_fingerprint(e2e_sessions[sender].aes_key) << "\n";
130
131 delete[] s_bytes; BN_free(modp); BN_free(peer_pub); BN_free(shared_secret);
132 BN_CTX_free(ctx);
133 }
134
135 // Message Decryption (Client)
136 else if (body.rfind("__E2E_MSG__ ", 0) == 0) {
137     std::string b64_cipher = body.substr(12);
138     std::string plaintext;
139     // Try Primary Key
140     if (e2e_sessions[sender].established && aes_decrypt(b64_cipher, e2e_sessions[
141 sender].aes_key, plaintext)) {
142         std::cout << "[" << get_timestamp() << "] [" << sender << "] (E2E): " <<
143 plaintext << "\n";
144     }
145     // Try Previous Key (For in-flight messages during a rotation)
146     else if (e2e_sessions[sender].has_old_key && aes_decrypt(b64_cipher, e2e_sessions[
147 sender].old_aes_key, plaintext)) {
148         std::cout << "[" << get_timestamp() << "] [" << sender << "] (E2E-Delayed): "
149 << plaintext << "\n";
150     } else {
151         std::cout << "[ERROR] Failed to decrypt E2E message from " << sender << ".\n";
152     }
153 }
154 }

```

Listing 6: Phase 5 Perfect Forward Secrecy (Key Rotation) Logic

## B LLM Prompts Used

Each phase of this project builds on the previous one, moving from a plaintext chat relay to a system with confidentiality, server authentication, end-to-end security independent of the server, and finally forward secrecy. Below I explain the design of each phase and the underlying networking/cryptographic reasoning that motivated it.

### B.1 Phase 1: Basic Client-Server Communication

The server is built around a single-threaded event loop using the POSIX `select()` system call rather than spawning a thread per client. This is a classic I/O multiplexing pattern: instead of blocking on one socket at a time, the process maintains an `fd_set` containing the listening socket, `STDIN_FILENO`, and every connected client socket, and asks the kernel which of these descriptors are actually ready for reading. On each loop iteration I rebuild the `fd_set` and recompute `max_sd` (the highest-numbered descriptor), since `select()` requires this range on every call. This avoids the overhead and synchronization complexity of multithreading while still handling many concurrent connections.

A core networking detail this phase has to address is that **TCP is a byte stream, not a message stream** – the kernel makes no guarantee that one `write()` on the sender’s side corresponds to one `read()` on the receiver’s side. A single `read()` call might return half of a logical message, or several messages concatenated together. To recover message boundaries I implement application-layer framing: each logical message is terminated with a newline (`\n`), and every socket has its own persistent receive buffer (`client_buffers` on the server, keyed by file descriptor; `rx_buffer` on the client). Incoming bytes are appended to the relevant buffer, and a loop repeatedly scans for `\n`, extracting and stripping any trailing `\r` from each complete frame while leaving any unterminated remainder in the buffer for the next read. On top of this framed transport I implemented registration, `/who`, `/chat <username>`, `@username <message>` routing, and `/quit`, with server-side audit logging of every relayed message for debugging and to make the plaintext nature of this phase visible before encryption is introduced.

### B.2 Phase 2: Diffie–Hellman Key Exchange and AES-256-GCM

Phase 1 sends everything in the clear, so Phase 2’s goal is to derive a shared secret between client and server over an untrusted network and use it to encrypt traffic. I use classic (finite-field) Diffie–Hellman with the standardized RFC 3526 MODP Group 15 (3072-bit) prime  $p$  and generator  $g = 2$ . The security of this exchange rests on the discrete logarithm problem: each side generates a private exponent uniformly at random

(`BN_rand_range()`), computes its public value  $A = g^a \bmod p$  (respectively  $B = g^b \bmod p$ ) using fast modular exponentiation (`BN_mod_exp`), and the two sides exchange  $A$  and  $B$  over the socket. Because exponentiation modulo  $p$  is easy in the forward direction but recovering  $a$  from  $A$  is believed to be computationally infeasible for a 3072-bit prime, an eavesdropper who only sees  $A$  and  $B$  cannot feasibly recover  $a$ ,  $b$ , or the shared secret. Each side then computes the same value  $K = g^{ab} \bmod p = A^b \bmod p = B^a \bmod p$ .

$K$  itself is a large integer, not a uniformly random bit string suitable directly as a symmetric key, so I run its raw bytes (`BN_bn2bin()`) through SHA-256 as a key derivation step. This serves two purposes: it compresses/normalizes the DH secret into a fixed 32-byte key with good statistical properties, and it means an attacker would need to break the hash function, not just guess at the integer's structure, to recover the key. Printing the first 8 bytes of that hash as a fingerprint on both ends is a debugging aid to confirm both sides converged on the identical key without ever transmitting the key itself.

For the actual message encryption I use AES-256 in Galois/Counter Mode (GCM) via OpenSSL's `EVP_CIPHER_CTX` interface. GCM is an authenticated encryption (AEAD) mode: alongside confidentiality (via CTR-mode-style encryption) it produces a 16-byte authentication tag computed over the ciphertext, so the receiver can detect any bit-flipping or truncation of the message, not just decrypt it. Each message gets a fresh random 12-byte IV/nonce (`RAND_bytes()`) – reusing a nonce with the same key under GCM is catastrophic, since it lets an attacker recover the authentication key via XOR of the two keystreams, so a fresh nonce per message is a hard requirement, not a style choice. The wire format I settled on is `IV || TAG || CIPHERTEXT`, Base64-encoded so it survives the newline-delimited text framing from Phase 1. On the decrypting side, the expected tag is supplied to the context via `EVP_CTRL_GCM_SET_TAG` before `EVP_DecryptFinal_ex()` is called; if the tag doesn't match, that call fails and no plaintext is released. The `tamper_client.cpp` demonstration exploits this directly: it flips a single bit in an intercepted ciphertext and shows that GCM's integrity check rejects it outright, rather than silently producing corrupted plaintext the way an unauthenticated cipher mode would.

What Phase 2 does *not* solve is authentication of the parties themselves – raw Diffie–Hellman is vulnerable to a man-in-the-middle who sits between client and server, runs a separate DH exchange with each side, and transparently relays (and reads) everything. That gap motivates Phase 3.

### B.3 Phase 3: Server Authentication via PKI

The MITM weakness in Phase 2 exists because neither side has any way to bind the DH public value it receives to a specific, verified identity. Phase 3 closes this gap with a small Public Key Infrastructure: I bootstrap a self-signed root CA (`ca_key.pem/ca_cert.pem`),

generate a server keypair, create a CSR binding the server's identity (`/CN=10.0.2.10`) to its public key, and sign that CSR with the CA key to produce `server_cert.pem`. This mirrors the standard X.509 trust model: a certificate is really just a public key plus identity metadata, digitally signed by an authority the verifier has chosen to trust in advance.

On connection, the server sends its certificate; the client loads it with `PEM_read_bio_X509()`, builds an `X509_STORE` seeded with the locally-trusted `ca_cert.pem`, and calls `X509_verify_cert()`. This walks the signature chain back to the trusted root, checks the CA's signature over the certificate body, and checks validity dates – if any of that fails, the certificate is rejected before any DH parameters are even exchanged. I additionally check that the certificate's Subject CN matches the server's expected address, since a validly-signed certificate for the *wrong* identity is a different attack (a rogue but CA-trusted server), not solved by chain validation alone.

Possessing a certificate is not the same as possessing the private key that matches it – an attacker could have simply copied the public `server_cert.pem` file. So Phase 3 adds a proof-of-possession step: the client generates a random 32-byte challenge (`RAND_bytes()`) and sends it to the server; the server signs it with its private key using `EVP_DigestSignInit()/EVP_DigestSignFinal()`; the client verifies that signature using the *public* key extracted from the already-validated certificate (`X509_get_pubkey()`) via `EVP_DigestVerifyInit()/EVP_DigestVerifyFinal()`. Only a party holding the private key can produce a signature that verifies under that public key, so this defeats an attacker who has stolen the certificate file but not the key. The two attack simulations (`mitm_proxy_invalid_cert.cpp` and `mitm_proxy_stolen_cert.cpp`) exist specifically to demonstrate each half of this: chain validation catching an untrusted self-signed certificate, and the challenge-response step catching a valid-but-stolen one. Only once both checks succeed does the client proceed to the Phase 2 Diffie–Hellman exchange, now anchored to a verified identity.

## B.4 Phase 4: End-to-End Encryption Between Clients

Phases 2 and 3 secure the link between each client and the server, but the server itself still sees plaintext – it decrypts, routes, and re-encrypts. Phase 4's goal is confidentiality that holds even if the server itself is compromised or curious, which requires a second, independent key exchange that happens *between* the two clients, using the server purely as a relay for opaque bytes.

Triggered by `/e2e <username>`, the initiating client runs its own ephemeral Diffie–Hellman exchange (same MODP 3072-bit group as Phase 2, but a fresh keypair, unrelated to the client-server session key) and sends its public value wrapped in a recognizable wire tag, `_E2E_INIT_`. The responder recognizes the tag, generates its own ephemeral key-



pair, computes the shared secret, derives a session key via SHA-256 exactly as in Phase 2, and replies with its own public value under an `__E2E_ACK__` tag. Both of these handshake messages travel *inside* the already-encrypted Phase 3 client-server channel, so the server can't read the DH public values either, though it could in principle see the tags. Once both sides compute the shared secret, they have a session key the server never sees or derives.

For actual chat content, I use double encryption: the plaintext is AES-GCM-encrypted under the E2E session key and Base64-encoded (tagged `__E2E_MSG__`), and that entire blob is then sent as the payload of a normal `@username <payload>` message, which the outer Phase 3 layer encrypts again for the client-server hop. The server's routing logic is unchanged – it decrypts the outer layer, sees an opaque Base64 string it has no key for, and relays it, which is exactly the point: the audit log at the server shows only ciphertext bytes and routing metadata, never plaintext. On the receiving client, the outer layer is peeled off first, then the inner `__E2E_MSG__` payload is decrypted with the E2E session key and displayed with an explicit (E2E) marker so it's visually distinct from server-only-encrypted traffic.

## B.5 Phase 5: Perfect Forward Secrecy via Key Rotation

A static session key, however well it was derived, has one structural weakness: if it is ever compromised (or the private DH exponents that produced it are somehow recovered later), *every* message ever encrypted under it becomes retroactively readable. Perfect Forward Secrecy addresses this by ensuring that compromising today's key tells an attacker nothing about yesterday's or tomorrow's traffic – which requires keys to actually change over time, and change in a way that isn't derivable from prior key material.

I implement this as time-boxed key rotation. The `E2ESession` struct tracks `last_rekey_time`, a rotation state machine, a pending ephemeral keypair (`next_priv_key`) generated in advance of completing a rotation, and a one-generation-back key (`old_aes_key/has_old_key`) to avoid dropping messages that were in flight when a rotation completed. Because the main client loop already blocks in `select()`, I give that call a short (1-second) timeout via `struct timeval tv` so the loop periodically wakes up on its own to check the rotation clock even with no network activity. Every 60 seconds, a client generates a brand-new ephemeral DH keypair – with fresh randomness, independent of the previous one – enters an INITIATED state, and sends an encrypted `__E2E_REKEY_INIT__` tag containing its new public value to its peer.

Because either side can independently decide it's time to rotate, both clients can trigger a rotation within the same round-trip – a "glare" condition. I resolve this deterministically rather than with additional network round-trips: if a client receives a `__E2E_REKEY_INIT__`

while it is itself already **INITIATED**, both sides compare usernames lexicographically. The client with the lexicographically larger ("dictator") username discards the incoming request and waits for its own to be acknowledged; the other ("yield") client abandons its own pending rotation, accepts the peer's public value, computes the new shared secret from it, and replies with `__E2E_REKEY_ACK__`. Because both sides apply the same comparison to the same two usernames, they always agree on which one wins, without needing to negotiate.

Each completed rotation derives a new AES key via SHA-256 over the new DH shared secret – exactly as in the initial handshake, so the new key is cryptographically unrelated to the old one – and the previous key is kept for a short grace period as `old_aes_key` rather than discarded immediately. Message decryption first tries the current key; if that fails (because the message was encrypted just before rotation completed but arrived just after), it falls back to `old_aes_key` and logs the message as (E2E-Delayed) rather than dropping it. Once a key is fully retired, however, it is not retained anywhere, and since each new key comes from fresh DH randomness rather than any transformation of the old key, recovering a later session key gives an attacker no path back to earlier ones – this is what gives the scheme forward secrecy: compromise of the current key exposes only the current 60-second window, not the conversation's history.